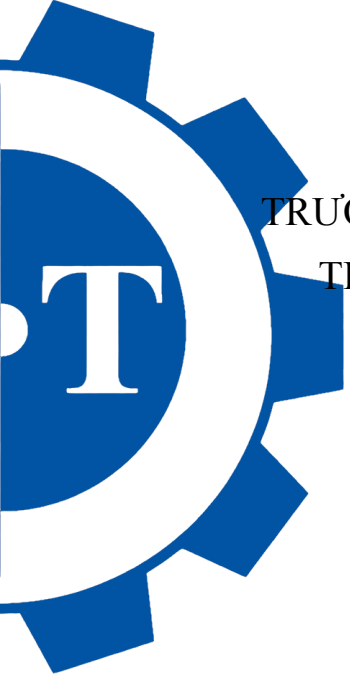




TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG HCM
TRUNG TÂM PHÁT TRIỂN CÔNG NGHỆ THÔNG TIN
Lớp: CS106.F31.CN2.TTNT

ĐỒ ÁN CUỐI KỲ
MÔN: TRÍ TUỆ NHÂN TẠO

**HỆ THỐNG PHÁT HIỆN GIAO DỊCH TÀI CHÍNH
BẤT THƯỜNG VÀ NGHI VẤN GIAN LẬN**

GVHD: Nguyễn Đình Hiền

Nhóm: 5

Tên sinh viên	MSSV
Ngô Thị Vân Giang	25730020
Lê Vân Anh	25730008
Bùi Thi Hoàng	25730029
Huỳnh Trường An	25730004
Trương Nam Anh	25730010



Mục lục

1	GIỚI THIỆU	1
1.1	Đặt vấn đề	1
1.2	Mục tiêu của đề tài	1
1.3	Phạm vi phân tích	2
2	PHƯƠNG PHÁP THU THẬP VÀ TIỀN XỬ LÝ DỮ LIỆU	3
2.1	Thu thập dữ liệu giao dịch	3
2.2	Mô tả bộ dữ liệu	3
2.3	Tiền xử lý dữ liệu	5
3	PHÂN TÍCH KHÁM PHÁ DỮ LIỆU	8
3.1	Phân bố dữ liệu	8
3.2	Gian lận theo loại hình dịch vụ	8
3.3	Giá trị bất thường	9
3.4	Quan hệ giữa các đặc trưng	10
4	XÂY DỰNG CÁC MÔ HÌNH HỌC MÁY	12
4.1	Chuẩn bị dữ liệu cho mô hình	12
4.2	Các chỉ số đánh giá mô hình	14
4.3	Xử lý dữ liệu mất cân bằng	15
4.4	Lựa chọn ngưỡng quyết định	15
4.5	Mô hình Random Forest	15
4.6	Mô hình XGBoost	16
4.7	Mô hình LightGBM	16
4.8	Mô hình CatBoost	18
4.9	Kết hợp mô hình bằng Stacking	18
4.10	Quá trình tinh chỉnh siêu tham số	18
5	ĐÁNH GIÁ VÀ THẢO LUẬN KẾT QUẢ	20
5.1	Phạm vi sáu thử nghiệm	20
5.2	Kết quả từng thử nghiệm	21
5.3	Giải thích ảnh hưởng của các đặc trưng	23

5.4	Tổng hợp và so sánh sáu thử nghiệm	25
5.5	Ma trận nhầm lẫn và đường Precision–Recall	26
5.6	Lựa chọn mô hình phù hợp với bộ dữ liệu	29
5.7	Đánh giá tính khả thi và hạn chế	29
5.8	Hướng phát triển trong tương lai	30
6	KẾT LUẬN	32

1.1 Đặt vấn đề

Sự phát triển nhanh chóng của các hình thức thanh toán điện tử và thẻ tín dụng giúp việc giao dịch trở nên thuận tiện, nhưng đồng thời cũng kéo theo nguy cơ gian lận ngày càng gia tăng. Các giao dịch gian lận gây thiệt hại lớn cho ngân hàng, đơn vị thanh toán và chính khách hàng [1], do đó việc phát hiện kịp thời các giao dịch bất thường là một bài toán quan trọng trong lĩnh vực tài chính.

Đặc điểm nổi bật của bài toán phát hiện gian lận là dữ liệu thường mất cân bằng nghiêm trọng: số giao dịch gian lận chỉ chiếm một tỷ lệ rất nhỏ so với giao dịch bình thường [2, 3]. Một mô hình đơn giản dự đoán tất cả giao dịch là hợp lệ vẫn có thể đạt Accuracy rất cao nhưng hoàn toàn bỏ sót gian lận. Vì vậy, bài toán cần được đánh giá bằng các chỉ số phù hợp như Recall, F2-Score hay PR-AUC thay vì chỉ dựa vào Accuracy [4].

Trong khuôn khổ môn học Trí tuệ nhân tạo, đề tài xây dựng một hệ thống học máy có khả năng phân loại một giao dịch tài chính thành hai nhóm: giao dịch hợp lệ (nhãn 0) và giao dịch gian lận (nhãn 1). Toàn bộ quy trình từ nạp dữ liệu, làm sạch, phân tích khám phá, tạo đặc trưng, xây dựng và đánh giá mô hình được thực hiện một cách bài bản trên cùng một bộ dữ liệu mô phỏng giao dịch thẻ tín dụng.

1.2 Mục tiêu của đề tài

Đề tài hướng tới các mục tiêu chính sau đây:

- Xây dựng quy trình xử lý hoàn chỉnh cho dữ liệu giao dịch thẻ tín dụng: nạp dữ liệu, làm sạch, xử lý dữ liệu thiếu, tạo đặc trưng và chuẩn hóa dữ liệu.
- Phân tích khám phá dữ liệu nhằm nhận diện các đặc điểm của giao dịch gian lận như loại hình dịch vụ có rủi ro cao, giá trị bất thường của số tiền giao dịch.
- Xây dựng và huấn luyện năm thuật toán phân loại: Random Forest, XGBoost (hai cấu hình), LightGBM và CatBoost, kết hợp các kỹ thuật xử lý dữ liệu mất cân bằng như `class_weight`, `scale_pos_weight`, `auto_class_weights` và SMOTE nhẹ; sau đó tích hợp các mô hình tốt nhất bằng kỹ thuật xếp chồng `StackingClassifier`.

- Thiết kế sáu thử nghiệm có kiểm soát để đánh giá ảnh hưởng của phương pháp biểu diễn đặc trưng (đặc trưng gốc, OneHotEncoder, TargetEncoder) và của việc loại bỏ các đặc trưng chứa nhiễu, trong khi giữ nguyên quy trình chia tập và tinh chỉnh siêu tham số.
- Đánh giá và so sánh hiệu suất các mô hình dựa trên các chỉ số Precision, Recall, F1-Score, F2-Score, ROC-AUC, PR-AUC, FNR và FPR; lựa chọn mô hình đề xuất cuối cùng.

1.3 Phạm vi phân tích

Bộ dữ liệu của đề tài gồm 30 058 giao dịch thẻ tín dụng mô phỏng được ghi nhận trong giai đoạn từ ngày 21/06 đến ngày 30/06/2020, với 23 cột thuộc tính mô tả thời gian giao dịch, thẻ, đơn vị chấp nhận thanh toán, loại hình dịch vụ, số tiền, vị trí địa lý và nhãn gian lận. Trong đó có 29 925 giao dịch hợp lệ và 133 giao dịch gian lận, khiến lớp gian lận nhỏ hơn lớp hợp lệ 225 lần.

Đề tài tập trung vào việc xây dựng và so sánh năm thuật toán Random Forest, XGBoost, LightGBM, CatBoost và mô hình xếp chồng Stacking trên cùng một bộ dữ liệu, cùng bốn chiến lược xử lý mất cân bằng lớp (điều chỉnh trọng số lớp, gán trọng số cho lớp thiểu số, trọng số tự động cân bằng và sinh dữ liệu tổng hợp SMOTE nhẹ). Các vấn đề về triển khai hệ thống phát hiện gian lận theo thời gian thực hay tích hợp nghiệp vụ ngân hàng không thuộc phạm vi nghiên cứu của đồ án.

2.1 Thu thập dữ liệu giao dịch

Bộ dữ liệu được sử dụng trong đồ án là tập dữ liệu giao dịch thẻ tín dụng mô phỏng, gồm 30 058 giao dịch và 23 cột thuộc tính. Dữ liệu được lưu trữ trên Google Sheets và được nạp trực tiếp vào chương trình bằng thư viện pandas [5] dưới dạng CSV. Quá trình nạp dữ liệu và làm sạch dữ liệu lần lượt được trình bày trong Hình 2.1 và Hình 2.2.

```
In [ ]:  
import io  
import urllib.request  
  
def _load_data_source(source, **read_kwargs):  
    """Nạp CSV từ Google Sheets; nếu mạng lỗi thì dùng file local 'sheet.csv'."""  
    try:  
        with urllib.request.urlopen(source, timeout=30) as response:  
            return pd.read_csv(io.BytesIO(response.read()), **read_kwargs)  
    except Exception:  
        return pd.read_csv("sheet.csv", **read_kwargs)  
  
sheet_id = "1ByffoXqSwnG8qHj4zaSki07hIIQS-43fR5mY8aquWnE"  
gid = "701522832"  
url = f"https://docs.google.com/spreadsheets/d/{sheet_id}/export?format=csv&gid={gid}"  
df = _load_data_source(url)  
df_raw = df.copy(deep=True)  
df_str = _load_data_source(url, dtype=str, keep_default_na=False)  
  
print(f"Số mẫu: {df.shape[0]} | Số cột: {df.shape[1]}")  
assert df.shape[0] >= 500, "Chưa đạt tối thiểu 500 mẫu theo yêu cầu đồ án"  
df
```

Hình 2.1: Đoạn chương trình nạp dữ liệu từ Google Sheets

2.2 Mô tả bộ dữ liệu

Bộ dữ liệu mô tả hoạt động giao dịch của 909 thẻ tín dụng tại 693 điểm chấp nhận thanh toán thuộc 50 bang của Hoa Kỳ, trong khoảng thời gian 10 ngày, từ ngày 21/06 đến ngày 30/06/2020. Mỗi bản ghi tương ứng với một giao dịch. Số tiền giao dịch dao động từ 1 USD đến 6 600,44 USD, với trung vị 46,28 USD. Bảng 2.1 liệt kê chi tiết các cột quan trọng của bộ dữ liệu.

```

In [ ]:

print('MỤC TIÊU: sửa các lỗi dữ liệu cơ bản trước khi phân tích.')

df = df_raw.copy(deep=True)
duplicate_count = int(df.duplicated().sum())
df = df.drop_duplicates().copy()
print(f'1. Đã loại {duplicate_count} dòng trùng; còn {len(df):,} dòng.')

def convert_to_number(value):
    if pd.isna(value):
        return np.nan
    text = re.sub(r'[^\d.,-]', '', str(value).strip())
    if not text or text in {'-', '.', ','}:
        return np.nan
    if '.' in text and ',' in text:
        text = (
            text.replace('.', ',')
            if text.rfind('.') > text.rfind(',')
            else text.replace('-', '.').replace('.', ',')
        )
    elif ',' in text:
        text = text.replace('.', ',') if text.count(',') > 1 else text.replace('.', ',')
    elif text.count('.') > 1:
        text = text.replace('.', ',')
    try:
        return float(text)
    except ValueError:
        return np.nan

for column in ['lat', 'long', 'amt', 'merch_lat', 'merch_long']:
    df[column] = df[column].apply(convert_to_number)
print('2. Đã chuyển lat, long và amt về dạng số.')

df['trans_date_trans_time'] = pd.to_datetime(
    df['trans_date_trans_time'], errors='coerce'
)
print('3. Đã chuyển trans_date_trans_time về dạng thời gian.')

def _repair_coordinate(raw, lower, upper):
    """Khôi phục tọa độ từ chuỗi bị mất dấu thập phân.

    raw: giá trị góc dạng chuỗi (vd '339.659' hoặc '-809.355').
    lower/upper: khoảng trị tuyệt đối hợp lệ của tọa độ cần khôi phục.
    """
    text = str(raw).strip()
    if text in ("", "nan", "NaN", "None"):
        return np.nan
    sign = -1.0 if text.startswith("-") else 1.0
    digits = "".join(ch for ch in text if ch.isdigit())
    if not digits:
        return np.nan
    value = int(digits)
    for scale in range(0, 9):
        candidate = value / (10.0 ** scale)
        if lower <= candidate <= upper:
            return sign * candidate
    return np.nan

df["lat"] = df_str["lat"].apply(lambda v: _repair_coordinate(v, 15.0, 75.0))
df["long"] = df_str["long"].apply(lambda v: _repair_coordinate(v, 66.0, 180.0))
df["merch_lat"] = df_str["merch_lat"].apply(lambda v: _repair_coordinate(v, 15.0, 75.0))
df["merch_long"] = df_str["merch_long"].apply(lambda v: _repair_coordinate(v, 66.0, 180.0))

card_identity = (
    df_str["first"] + "|" + df_str["last"] + "|" + df_str["gender"] + "|"
    + df_str["street"] + "|" + df_str["city"] + "|" + df_str["state"] + "|"
    + df_str["job"] + "|" + df_str["dob"]
)
df["cc_num"] = pd.factorize(card_identity)[0]

df["unix_time"] = (
    df["trans_date_trans_time"] - pd.Timestamp("1970-01-01")
).dt.total_seconds().astype("int64")

recovered_cards = int(df["cc_num"].nunique())
print(f'2b. Đã sửa dữ liệu nguồn: khôi phục tọa độ, gán lại mã thẻ ({recovered_cards:,} thẻ), tính lại unix_time.')

required_columns = [
    'lat', 'long', 'amt', 'trans_date_trans_time',
    'cc_num', 'category', 'is_fraud'
]
missing_summary = df[required_columns].isna().sum()
missing_rows = int(df[required_columns].isna().any(axis=1).sum())
print(f'4. Có {missing_rows:,} dòng thiếu ít nhất một cột bắt buộc.')
if missing_rows > 0:
    display(missing_summary[missing_summary > 0])
    df = df.dropna(subset=required_columns).copy()
else:
    print('Không có dòng thiếu dữ liệu bắt buộc.')

```

Hình 2.2: Đoạn chương trình làm sạch dữ liệu

Đặc trưng	Kiểu dữ liệu	Mô tả
trans_date_trans_time	datetime	Thời điểm giao dịch
cc_num	string	Mã thẻ tín dụng
merchant	string	Đơn vị chấp nhận thanh toán
category	string	Loại hình dịch vụ như mua sắm, ăn uống, du lịch
amt	float	Số tiền giao dịch (USD)
gender	string	Giới tính chủ thẻ
city / state	string	Thành phố / bang nơi giao dịch
lat / long	float	Vĩ độ / kinh độ vị trí thường trú của chủ thẻ
city_pop	int	Dân số thành phố
job / dob	string	Nghề nghiệp / ngày sinh chủ thẻ
trans_num / unix_time	string	Mã giao dịch / thời gian dạng epoch
merch_lat / merch_long	float	Vị trí của đơn vị chấp nhận thanh toán
is_fraud	int	Nhãn: 0 = hợp lệ, 1 = gian lận

Bảng 2.1: Các đặc trưng của bộ dữ liệu

Phân bố nhãn của bộ dữ liệu cho thấy tính mất cân bằng nghiêm trọng: có 29 925 giao dịch hợp lệ nhưng chỉ có 133 giao dịch gian lận, tương ứng tỷ lệ gian lận 0,44%. Lớp lớn gấp 225 lần lớp nhỏ. Vì vậy, Accuracy không phản ánh đúng khả năng phát hiện gian lận của mô hình và không được dùng làm tiêu chí đánh giá chính trong đề án.

2.3 Tiền xử lý dữ liệu

2.3.1 Làm sạch dữ liệu

Các cột số như amt, lat, long, merch_lat, merch_long có thể chứa ký tự hoặc dấu phân cách không mong muốn, được chuẩn hóa về dạng số thực bằng hàm `convert_to_number`. Cột `trans_date_trans_time` được chuyển sang kiểu `datetime` để có thể trích xuất giờ giao dịch. Kiểm tra cho thấy bộ dữ liệu không có dòng trùng lặp hoàn toàn và không có dòng thiếu các cột bắt buộc gồm lat, long, amt, trans_date_trans_time, cc_num, category và is_fraud; toàn bộ 30 058 giao dịch được giữ lại cho các bước phân tích tiếp theo.

Quá trình kiểm tra dữ liệu nguồn phát hiện ba lỗi phát sinh khi bộ dữ liệu được nhập vào Google Sheets, được khôi phục trước khi phân tích: (1) các cột tọa độ lat, long, merch_lat, merch_long bị mất dấu chấm thập phân (ví dụ 33,9659 bị nhập thành 339.659), được khôi phục bằng cách tách toàn bộ chữ số rồi dịch dấu thập phân về bậc sao cho giá trị rơi vào khoảng hợp lệ của vĩ độ (15–75) và kinh độ (66–180); (2) cột mã thẻ cc_num bị làm tròn về ba chữ số có nghĩa nên chỉ còn 309 mã khác nhau, được thay bằng định danh thẻ ổn định suy ra từ định danh chủ thẻ (họ, tên, địa chỉ, thành phố, bang, nghề nghiệp, ngày sinh), thu được 909 thẻ; (3) cột

unix_time bị ghi đè toàn bộ bằng cùng một hằng số, được tính lại từ cột trans_date_trans_time.

2.3.2 Tạo đặc trưng

Từ dữ liệu thô, một số đặc trưng hành vi được tạo thêm để phục vụ mô hình hóa, minh họa trong Hình 2.3. Danh sách 14 đặc trưng đầu vào gồm:

- amt, age, hour: số tiền giao dịch, tuổi chủ thẻ và giờ trong ngày của giao dịch.
- amt_vs_card_median, amt_vs_card_avg: chênh lệch và tỷ lệ giữa số tiền giao dịch với trung vị, trung bình số tiền của thẻ, phản ánh giao dịch lệch thói quen chi tiêu.
- txn_freq_hour, txn_count_recent: tần suất giao dịch của cùng một thẻ trong một giờ và số giao dịch gần đây, phản ánh hoạt động bất thường.
- distance_km, speed_kmh: khoảng cách từ vị trí của đơn vị chấp nhận thanh toán đến vị trí thường trú của thẻ (tính theo công thức Haversine) và tốc độ di chuyển giữa giao dịch hiện tại với giao dịch trước của cùng thẻ. Vì tọa độ lat/long cố định theo từng thẻ, khoảng cách này phản ánh việc giao dịch ở nơi xa nơi ở quen thuộc.
- amt_log, amt_zscore_card: logarit số tiền và mức độ lệch của số tiền so với trung bình thẻ theo độ lệch chuẩn.
- amt_per_km: tỷ lệ giữa số tiền giao dịch và khoảng cách di chuyển.
- is_night: cờ cho biết giao dịch xảy ra vào ban đêm (từ 22 giờ đến 5 giờ).
- category: loại hình dịch vụ.

Các đặc trưng thống kê theo thẻ (card profile) chỉ được ước lượng trên tập huấn luyện, còn các đặc trưng phụ thuộc lịch sử riêng của thẻ như speed_kmh, txn_count_recent được tính riêng trong từng tập, nhằm tránh rò rỉ thông tin từ tập kiểm tra vào quá trình huấn luyện.

```

In [ ]:
selected_training_features = [
    'amt', 'age', 'amt_vs_card_median', 'hour',
    'txn_freq_hour', 'distance_km', 'category',
    'amt_log', 'amt_zscore_card', 'is_night',
    'speed_kmh', 'amt_per_kmh',
    'txn_count_recent', 'amt_vs_card_avg',
]

raw_feature_cols = ['cc_num', 'amt', 'lat', 'long', 'merch_lat', 'merch_long',
                    'trans_date_trans_time', 'dob', 'category', 'trans_num', 'is_fraud']

def fit_card_profile(train_raw):
    """Fit hồ sơ thẻ CHI từ tập train (median/mean/std của amt, tần suất theo giờ)."""
    hour_col = train_raw['trans_date_trans_time'].dt.hour
    profile = {
        'amt_median': train_raw.groupby('cc_num')['amt'].median(),
        'amt_mean': train_raw.groupby('cc_num')['amt'].mean(),
        'amt_std': train_raw.groupby('cc_num')['amt'].std().fillna(0.0),
        'hour_freq': train_raw.assign(hour=hour_col).groupby(['cc_num', 'hour']).size(),
        'global_median': train_raw['amt'].median(),
        'global_mean': train_raw['amt'].mean(),
        'global_std': train_raw['amt'].std(),
    }
    return profile

def build_features(part_raw, card_profile):
    """Build 14 features cho MỘT tập, dùng hồ sơ thẻ fit từ TRAIN (không rò rỉ test)."""
    part = part_raw.copy()
    part = part.sort_values('trans_date_trans_time') # sắp theo thời gian, giữ nguyên index
    part['hour'] = part['trans_date_trans_time'].dt.hour

    med = part['cc_num'].map(card_profile['amt_median']).fillna(card_profile['global_median'])
    mean = part['cc_num'].map(card_profile['amt_mean']).fillna(card_profile['global_mean'])
    std = part['cc_num'].map(card_profile['amt_std']).fillna(card_profile['global_std'])

    part['age'] = (part['trans_date_trans_time'].dt.year
                  - pd.to_datetime(part['dob'], errors='coerce').dt.year)
    part['amt_vs_card_median'] = part['amt'] - med
    part['txn_freq_hour'] = part[['cc_num', 'hour']].apply(
        lambda r: int(card_profile['hour_freq'].get((r['cc_num'], r['hour']), 0)), axis=1
    )
    part['amt_log'] = np.log1p(part['amt'])
    part['amt_zscore_card'] = (part['amt'] - mean) / std.clip(lower=0.01)
    part['is_night'] = part['hour'].apply(lambda h: 1 if h >= 22 or h < 5 else 0).astype(int)

    part['distance_km'] = calculate_distance(
        part['lat'], part['long'],
        part['merch_lat'], part['merch_long']
    )

    time_diff_h = (part.groupby('cc_num')['trans_date_trans_time'].diff()
                  .dt.total_seconds().fillna(3600) / 3600)
    part['speed_kmh'] = part['distance_km'] / time_diff_h.clip(lower=0.01)
    part['amt_per_kmh'] = part['amt'] / part['distance_km'].clip(lower=0.01)
    part['txn_count_recent'] = part.groupby('cc_num')['trans_num'].cumcount() + 1
    part['amt_vs_card_avg'] = part['amt'] / mean.clip(lower=0.01)

    return part[selected_training_features]

print(f'Số lượng feature: {len(selected_training_features)}')

```

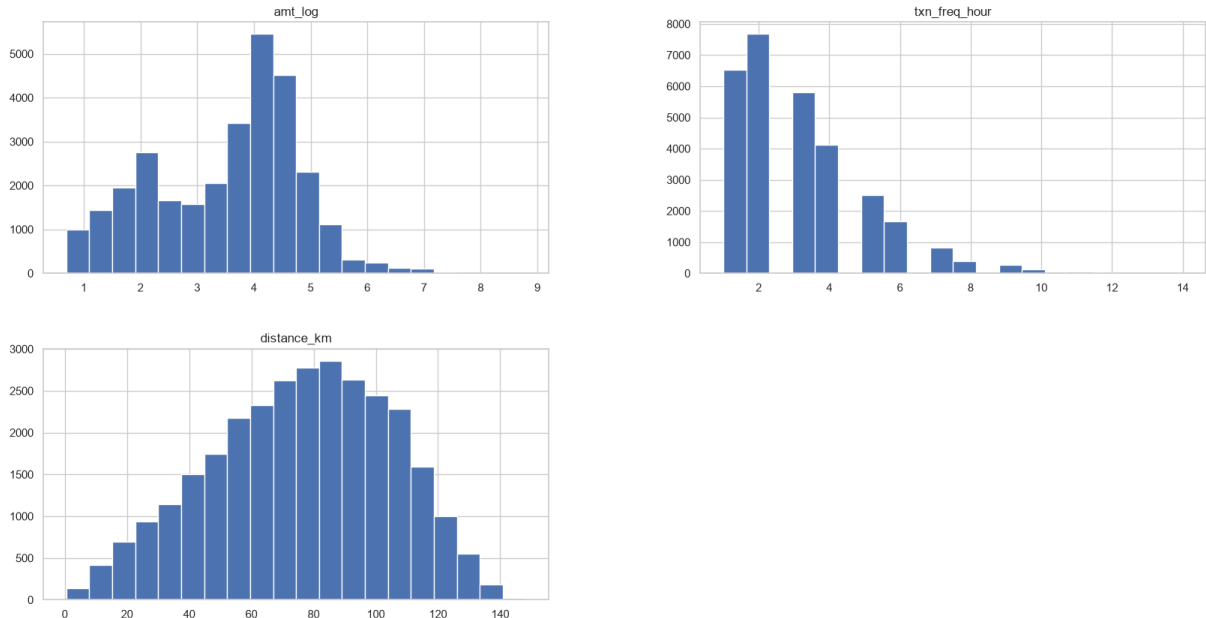
Hình 2.3: Đoạn chương trình tạo các đặc trưng hành vi

Tóm lại, bài toán phân loại giao dịch thẻ tín dụng được phát biểu như sau: đầu vào là một giao dịch được mô tả bằng 14 đặc trưng gồm các giá trị số như số tiền, tuổi, giờ giao dịch, tần suất giao dịch, khoảng cách và tốc độ di chuyển, cũng như loại hình dịch vụ; đầu ra là nhãn phân loại 0 (giao dịch hợp lệ) hoặc 1 (giao dịch gian lận). Mô hình học máy trả về xác suất gian lận của từng giao dịch, sau đó ngưỡng quyết định được sử dụng để chuyển xác suất đó thành nhãn cuối cùng.

Phân tích khám phá giúp hiểu rõ cấu trúc dữ liệu, nhận diện các đặc điểm của giao dịch gian lận và phát hiện giá trị bất thường trước khi xây dựng mô hình. Quá trình này sử dụng các thư viện seaborn và matplotlib để trực quan hóa kết quả.

3.1 Phân bố dữ liệu

Hình 3.1 mô tả phân bố của ba đặc trưng số chính: logarit số tiền giao dịch `amt_log`, tần suất giao dịch `txn_freq_hour` và khoảng cách `distance_km`. Cả ba biến đều phân bố lệch về bên trái: phần lớn giao dịch có số tiền nhỏ, mỗi thẻ chỉ giao dịch một vài lần trong một giờ và khoảng cách từ đơn vị chấp nhận thanh toán đến vị trí thường trú của thẻ thường không quá 150 km. Phần đuôi dài bên phải của phân bố chứa các giá trị bất thường, nơi tập trung nhiều giao dịch gian lận.

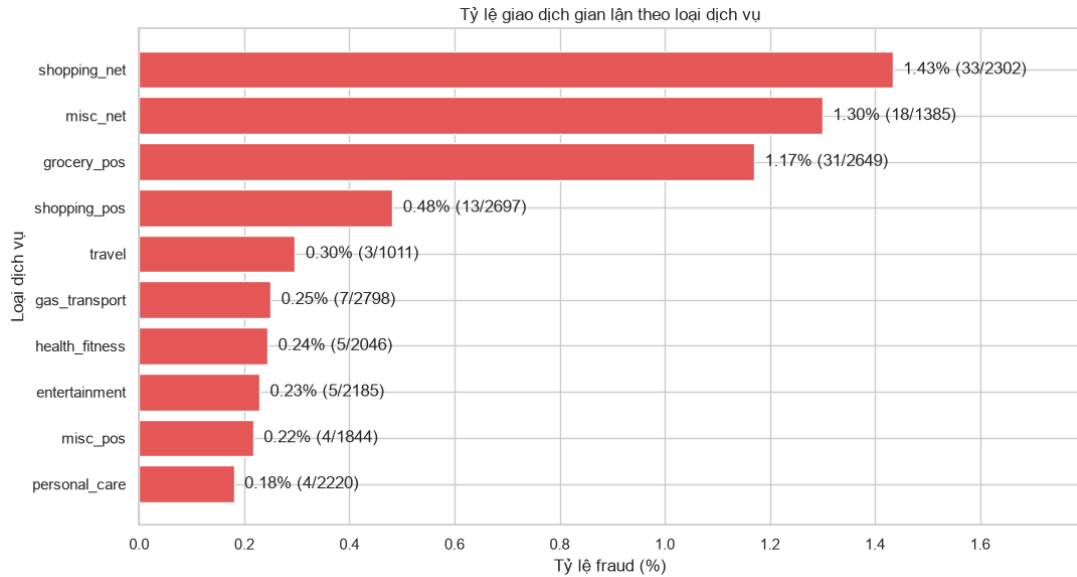


Hình 3.1: Phân bố của các đặc trưng số trong bộ dữ liệu

3.2 Gian lận theo loại hình dịch vụ

Hình 3.2 thể hiện tỷ lệ gian lận của 10 loại hình dịch vụ có rủi ro cao nhất. Ba nhóm đầu là `shopping_net`, `misc_net` và `grocery_pos` chiếm 82 trong tổng số 133 vụ gian lận, tương ứng

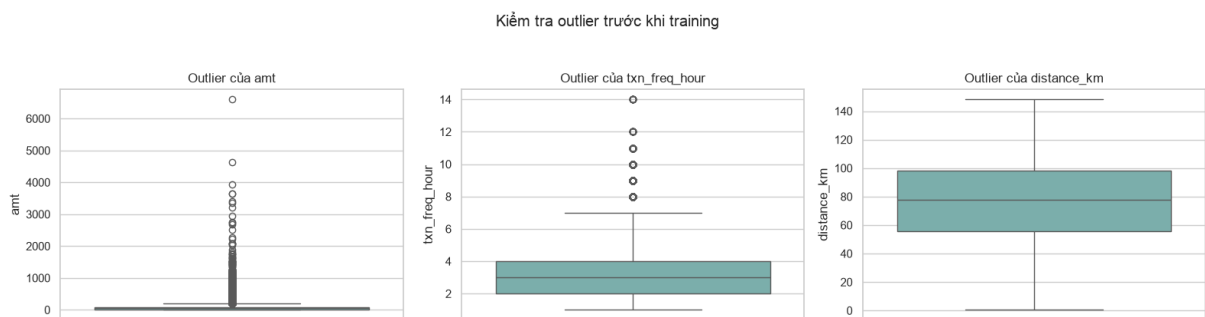
61,7%. Loại shopping_net có tỷ lệ gian lận cao nhất với 33 trong 2 302 giao dịch, tương ứng 1,43%, trong khi nhóm an toàn nhất personal_care chỉ có 4 trong 2 220 giao dịch, tương ứng 0,18%. Kết quả kiểm định Chi-square cho giá trị $p = 1,28 \times 10^{-23}$, khẳng định loại hình dịch vụ có liên quan có ý nghĩa thống kê với gian lận, do đó category được giữ lại làm đặc trưng cho mô hình.



Hình 3.2: Tỷ lệ giao dịch gian lận theo loại hình dịch vụ

3.3 Giá trị bất thường

Hình 3.3 dùng biểu đồ hộp để khảo sát giá trị bất thường của các đặc trưng số, và Bảng 3.1 thống kê chi tiết theo quy tắc $1,5 \times IQR$.



Hình 3.3: Biểu đồ hộp kiểm tra giá trị bất thường

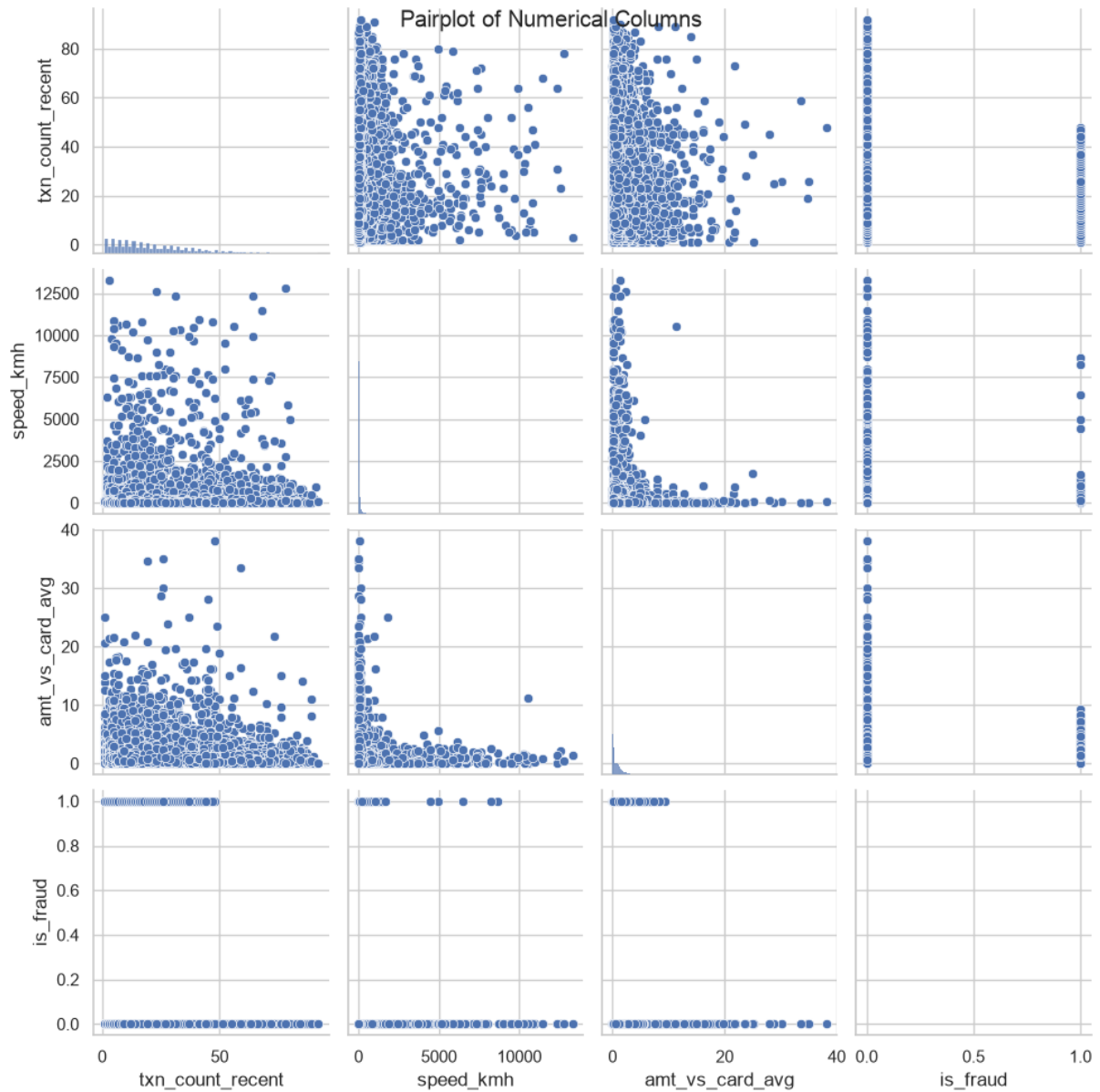
Đặc trưng	Ngưỡng trên	Số giá trị bất thường (tỷ lệ)	Gian lận trong vùng bất thường
amt	192,16	1 521 (5,06%)	104 (78,2%)
txn_freq_hour	7,00	887 (2,95%)	5
distance_km	162,10	0 (0%)	0

Bảng 3.1: Thống kê giá trị bất thường theo quy tắc IQR

Đáng chú ý, 78,2% số giao dịch gian lận nằm trong vùng giá trị bất thường của cột amt. Vì vậy, đồ án không xóa bỏ các giá trị bất thường ngay ở bước này mà chỉ giới hạn chúng theo quy tắc IQR sau khi chia tập huấn luyện và tập kiểm tra, nhằm tránh làm mất đi chính những giao dịch gian lận.

3.4 Quan hệ giữa các đặc trưng

Hình 3.4 trình bày quan hệ cặp giữa ba đặc trưng hành vi gồm số lần giao dịch gần đây txn_count_recent, tốc độ di chuyển speed_kmh và tỷ lệ số tiền so với trung bình thẻ amt_vs_card_avg, tô màu theo nhãn gian lận. Giao dịch gian lận có tốc độ di chuyển và tỷ lệ số tiền so với trung bình thẻ cao hơn rõ rệt: trung vị speed_kmh là 59,4 km/h so với 21,2 km/h của giao dịch hợp lệ, trung vị amt_vs_card_avg là 1,94 so với 0,67, cho thấy gian lận thường phát sinh ở nơi xa nơi ở quen thuộc và với số tiền vượt trung bình chi tiêu của thẻ. Ngược lại, txn_count_recent gần như không phân biệt hai lớp (trung vị 20 so với 19), nên chủ yếu đóng vai trò đặc trưng hỗ trợ.



Hình 3.4: Quan hệ cặp giữa các đặc trưng hành vi theo nhãn gian lận

Qua các phân tích trên, 14 đặc trưng được chọn để đưa vào mô hình gồm: `amt`, `age`, `amt_vs_card_median`, `hour`, `txn_freq_hour`, `distance_km`, `category`, `amt_log`, `amt_zscore_card`, `is_night`, `speed_kmh`, `amt_per_km`, `txn_count_recent` và `amt_vs_card_avg`. Các cột định danh như `cc_num` hay `trans_num` không được dùng trực tiếp vì mô hình có thể học thuộc mã thay vì học quy luật tổng quát của gian lận.

4.1 Chuẩn bị dữ liệu cho mô hình

4.1.1 Tách đặc trưng và biến mục tiêu

Từ bộ dữ liệu đã làm sạch, biến mục tiêu `is_fraud` là nhãn gian lận được tách khỏi các đặc trưng đầu vào. Mười bốn đặc trưng được giữ lại gồm mười ba đặc trưng số `amt`, `age`, `amt_vs_card_median`, `hour`, `txn_freq_hour`, `distance_km`, `amt_log`, `amt_zscore_card`, `is_night`, `speed_kmh`, `amt_per_km`, `txn_count_recent`, `amt_vs_card_avg` và đặc trưng phân loại `category`. Đặc trưng phân loại `category` được mã hóa bằng `TargetEncoder`, tức mã hóa theo tương quan với biến mục tiêu, còn các đặc trưng số được chuẩn hóa về trung bình 0 và độ lệch chuẩn 1 bằng `StandardScaler`. Hai bước này được gói trong một `ColumnTransformer` [6] để đồng bộ giữa tập huấn luyện và tập kiểm tra.

4.1.2 Chia tập dữ liệu

Bộ dữ liệu 30 058 giao dịch được chia theo tỷ lệ 80–20 bằng hàm `train_test_split` với `stratify = y` và `random_state = 42`, cho tập huấn luyện 24 046 mẫu, trong đó có 106 giao dịch gian lận, và tập kiểm tra 6 012 mẫu, trong đó có 27 giao dịch gian lận. Tập huấn luyện được chia tiếp thành hai phần: phần `Fit` gồm 19 236 mẫu, chiếm 80%, dùng để huấn luyện và tinh chỉnh mô hình; phần `Validation` gồm 4 810 mẫu, chiếm 20%, dùng riêng để chọn ngưỡng quyết định. Tập kiểm tra chỉ được dùng đúng một lần để đánh giá cuối cùng.

Để tránh rò rỉ dữ liệu, mọi quy tắc học từ dữ liệu đều được tính riêng trên tập huấn luyện rồi mới áp dụng sang tập kiểm tra: giới hạn xử lý giá trị bất thường được suy ra từ tập huấn luyện, `StandardScaler` và `TargetEncoder` chỉ được học trên tập huấn luyện. Các giá trị bất thường của đặc trưng số được giới hạn theo quy tắc IQR tính trên tập huấn luyện thay vì bị xóa, vì giá trị bất thường có thể chính là giao dịch gian lận. Hình 4.1 và Hình 4.2 trình bày đoạn chương trình chia dữ liệu và tiền xử lý.

```
In [ ]:  
  
y_full = df['is_fraud'].astype(int)  
df_raw_train, df_raw_test, y_train, y_test = train_test_split(  
    df[raw_feature_cols], y_full,  
    test_size=0.20, stratify=y_full, random_state=42  
)  
  
card_profile = fit_card_profile(df_raw_train)  
  
X_train_raw = build_features(df_raw_train, card_profile)  
X_test_raw = build_features(df_raw_test, card_profile)  
  
y_train = y_train.loc[X_train_raw.index].reset_index(drop=True)  
y_test = y_test.loc[X_test_raw.index].reset_index(drop=True)  
X_train_raw = X_train_raw.reset_index(drop=True)  
X_test_raw = X_test_raw.reset_index(drop=True)  
  
X_train_raw.shape  
  
  
cat_features = X_train_raw.select_dtypes(include=['object', 'category', 'string']).columns  
num_features = X_train_raw.select_dtypes(include=['number']).columns  
  
X_train_clean = X_train_raw.copy()  
iqr_bounds = {}  
for col in num_features:  
    q1, q3 = X_train_raw[col].quantile([0.25, 0.75])  
    iqr = q3 - q1  
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr  
    iqr_bounds[col] = (lower, upper)  
    X_train_clean[col] = X_train_clean[col].clip(lower, upper)  
  
print(f'IQR clipping applied to {len(num_features)} numeric features on TRAIN only.')  
print(f'Train shape: {X_train_clean.shape} | Test shape (untouched): {X_test_raw.shape}')
```

Hình 4.1: Đoạn chương trình chia tập dữ liệu và giới hạn giá trị bất thường

```

In [ ]:

y_train_array = np.asarray(y_train).ravel()
X_fit_clean, X_validation_clean, y_fit, y_validation = train_test_split(
    X_train_clean, y_train_array,
    test_size=0.20, stratify=y_train_array, random_state=42,
)

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), num_features),
        ('cat', TargetEncoder(), cat_features)
    ],
    remainder='drop'
)

X_fit_encoded = preprocessor.fit_transform(X_fit_clean, y_fit)

cat_feature_names = preprocessor.named_transformers_['cat'].get_feature_names_out(cat_features)
feature_names = list(num_features) + list(cat_feature_names)

if hasattr(X_fit_encoded, 'toarray'):
    X_fit_encoded = X_fit_encoded.toarray()

X_fit_encoded = pd.DataFrame(
    X_fit_encoded,
    columns=feature_names,
    index=X_fit_clean.index
)

X_validation_encoded = preprocessor.transform(X_validation_clean)
if hasattr(X_validation_encoded, 'toarray'):
    X_validation_encoded = X_validation_encoded.toarray()

X_validation_encoded = pd.DataFrame(
    X_validation_encoded,
    columns=feature_names,
    index=X_validation_clean.index
)

X_test_encoded = None # sẽ được gán sau

print('Thông kê Fit sau IQR clipping + StandardScaler:')
display(
    X_fit_encoded[num_features]
    .describe()
    .loc[['mean', 'std', 'min', 'max']]
    .round(2)
)

print(f'\nFit: {X_fit_encoded.shape}')
print(f'Validation: {X_validation_encoded.shape}')
print(f'Fit labels: {pd.Series(y_fit).value_counts().to_dict()}')
print('Test sẽ được transform sau khi hoàn tất tuning.')

```

Hình 4.2: Đoạn chương trình mã hóa và chuẩn hóa đặc trưng

4.2 Các chỉ số đánh giá mô hình

Do tính chất mất cân bằng nghiêm trọng của dữ liệu, mô hình được đánh giá bằng các chỉ số Precision, Recall, F1-Score, F2-Score, ROC-AUC và PR-AUC:

$$\begin{aligned}
 \text{Precision} &= \frac{TP}{TP + FP}, & \text{Recall} &= \frac{TP}{TP + FN}, \\
 F1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, & F_{\beta} &= (1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}},
 \end{aligned}$$

trong đó TP , FP , FN lần lượt là số giao dịch gian lận được phát hiện đúng, số giao dịch hợp lệ bị báo nhầm và số giao dịch gian lận bị bỏ sót. Ngoài ra, đồ án còn dùng ROC-AUC và PR-AUC; ROC-AUC đo khả năng phân biệt hai lớp theo xác suất, còn PR-AUC là diện tích dưới đường Precision–Recall và phù hợp hơn với dữ liệu mất cân bằng [4].

Bởi vì bỏ sót giao dịch gian lận thường gây thiệt hại lớn hơn nhiều so với kiểm tra nhầm một giao dịch hợp lệ, chỉ số F2-Score với $\beta = 2$ được ưu tiên làm tiêu chí chính: chỉ số này đặt trọng số Recall gấp đôi Precision, khuyến khích mô hình phát hiện được nhiều gian lận hơn.

4.3 Xử lý dữ liệu mất cân bằng

Dữ liệu của đề tài có tỷ lệ mất cân bằng trên tập Fit là $\frac{19151}{85} \approx 225:1$ (kết quả tính từ dữ liệu của đồ án). Vì vậy, đồ án áp dụng bốn chiến lược xử lý dữ liệu mất cân bằng — một vấn đề phổ biến trong mô hình dự đoán trên dữ liệu mất cân bằng [7] — tương ứng với năm cấu hình mô hình, được trình bày trong Bảng 4.1.

Chiến lược	Mô hình	Cách thiết lập
Trọng số lớp	Random Forest	<code>class_weight = "balanced"</code> hoặc trọng số thủ công $\{0:1, 1:k\}$ với $k \in \{20, 40, 60, 100, 150, 225\}$
Trọng số lớp thiểu số	XGBoost, LightGBM	<code>scale_pos_weight</code> đặt quanh tỷ lệ mất cân bằng, gồm các mức 0,10; 0,25; 0,50; 0,75 và 1,0 lần tỷ lệ 225
SMOTE nhẹ [8]	XGBoost	Sinh thêm mẫu tổng hợp cho lớp gian lận với tỷ lệ thiểu số/đa số chỉ từ 2% đến 5%, thay vì cân bằng hoàn toàn
Trọng số tự động cân bằng	CatBoost	<code>auto_class_weights = "Balanced"</code> , trọng số nghịch đảo với tần suất của từng lớp

Bảng 4.1: Bốn chiến lược xử lý dữ liệu mất cân bằng

Chiến lược SMOTE được đặt trong một Pipeline nhằm bảo đảm việc sinh dữ liệu tổng hợp chỉ diễn ra bên trong từng fold cross-validation, không làm rò rỉ thông tin vào tập xác thực.

4.4 Lựa chọn ngưỡng quyết định

Mô hình học máy trả về xác suất gian lận cho mỗi giao dịch. Thay vì dùng ngầm định ngưỡng 0,5, đồ án chọn ngưỡng tối ưu riêng cho từng mô hình trên tập Validation theo tiêu chí F2-Score cao nhất, đồng thời yêu cầu Precision tối thiểu 0,10 để tránh cảnh báo tràn lan. Cách tiếp cận này giúp ngưỡng được chọn độc lập với tập kiểm tra, và việc chỉ đánh giá một lần duy nhất trên tập kiểm tra giữ cho kết quả cuối cùng không bị lạc quan.

4.5 Mô hình Random Forest

Random Forest [9] xây dựng nhiều cây quyết định trên các mẫu bootstrap khác nhau và lấy biểu quyết từ các cây, giúp giảm phương sai và ít bị ảnh hưởng bởi nhiễu. Không gian siêu tham số được khảo sát được trình bày trong Bảng 4.2. Điều chỉnh trọng số lớp kết hợp với giới hạn độ sâu giúp các cây tập trung hơn vào lớp thiểu số mà không quá khớp nhiễu.

Tham số	Giá trị khảo sát
n_estimators	số cây, từ 400 đến 1 500
max_depth	None, 10, 15, 20, 25, 30, 40, 50
min_samples_split, min_samples_leaf	ràng buộc kích thước nút và lá
max_features	"sqrt", "log2", 0,3, 0,5, 0,7, 1,0
criterion	"gini", "entropy", "log_loss"
class_weight	"balanced", "balanced_subsample", {0:1, 1:k} với $k = 20, 40, 60, 100, 150, 225$

Bảng 4.2: Không gian siêu tham số của Random Forest

4.6 Mô hình XGBoost

XGBoost [10] là thuật toán gradient boosting xây dựng các cây tuần tự, cây sau được huấn luyện để khắc phục sai lệch của các cây trước, kèm cơ chế chính quy hóa gồm gamma, reg_alpha và reg_lambda để chống quá khớp. Không gian siêu tham số được khảo sát được trình bày trong Bảng 4.3.

Tham số	Giá trị khảo sát
n_estimators	số cây, từ 300 đến 1 500
max_depth	độ sâu tối đa, từ 2 đến 10
learning_rate	tốc độ học, từ 0,005 đến 0,15
subsample, colsample_bytree	tỷ lệ mẫu và đặc trưng dùng cho mỗi cây, từ 0,55 đến 1,00
min_child_weight	trọng số tối thiểu của nút con, từ 1 đến 20
gamma	ngưỡng suy giảm lỗi tối thiểu để tách nút, từ 0 đến 2
reg_alpha, reg_lambda	hệ số chính quy hóa L1 và L2
scale_pos_weight	trọng số lớp gian lận: 1; $\sqrt{225}$; 0,10; 0,25; 0,50; 0,75; 1,00 lần tỷ lệ mất cân bằng

Bảng 4.3: Không gian siêu tham số của XGBoost

4.7 Mô hình LightGBM

LightGBM [11] là thuật toán gradient boosting dùng bộ phân tách dựa trên hướng (histogram-based), cho phép huấn luyện nhanh trên dữ liệu lớn và tự xử lý mất cân bằng lớp bằng tham số scale_pos_weight [12]. Không gian siêu tham số được khảo sát được trình bày trong Bảng 4.4.

Tham số	Giá trị khảo sát
n_estimators	số cây, từ 300 đến 1 500
max_depth	độ sâu tối đa, từ 3 đến 12
num_leaves	số lá trên mỗi cây, từ 15 đến 64
learning_rate	tốc độ học, từ 0,005 đến 0,15
min_child_samples	số mẫu tối thiểu của một nút lá, từ 5 đến 50
subsample, colsample_bytree	tỷ lệ mẫu và đặc trưng dùng cho mỗi cây, từ 0,55 đến 1,00
reg_alpha, reg_lambda	hệ số chính quy hóa L1 và L2
scale_pos_weight	trọng số lớp gian lận: 1; $\sqrt{225}$; 0,25; 0,50; 0,75; 1,00 lần tỷ lệ mất cân bằng

Bảng 4.4: Không gian siêu tham số của LightGBM

4.7.1 Cấu hình tối ưu của LightGBM

Sau khi RandomizedSearchCV với 60 lần thử trên cross-validation 5-fold, LightGBM đạt điểm PR-AUC cross-validation cao nhất là 0,8525, vượt trội so với bốn cấu hình còn lại. Bộ siêu tham số tương ứng được trình bày ở Bảng 4.5. Cấu hình này dùng scale_pos_weight bằng 15,01, tức nhẹ hơn đáng kể so với tỷ lệ mất cân bằng đầy đủ là 225,31, cho thấy việc tăng trọng số lớp thiểu số vừa đủ giúp mô hình tập trung vào lớp gian lận mà không làm méo quá mức phân phối xác suất. Số cây 1 483 đi kèm tốc độ học thấp 0,0314 là một cấu hình hội tụ chậm, vốn là lựa chọn phổ biến cho bài toán dữ liệu nhỏ và mất cân bằng.

Tham số	Giá trị tối ưu
n_estimators	1 483
learning_rate	0,0314
num_leaves	21
max_depth	6
min_child_samples	17
subsample	0,8310
colsample_bytree	0,8220
reg_alpha	0,00281
reg_lambda	0,13136
scale_pos_weight	15,01

Bảng 4.5: Bộ siêu tham số tối ưu của LightGBM theo RandomizedSearchCV

Với cấu hình này, chênh lệch giữa điểm PR-AUC cross-validation trên tập Fit là 0,8525 và điểm PR-AUC trên tập kiểm tra là 0,8197 ở mức 0,0328, cho thấy mức độ quá khớp ở mức chấp nhận được và không cần dựng thêm biến thể giảm quá khớp. Kết quả chi tiết của cấu hình này trên tập kiểm tra được trình bày ở Bảng 5.4 của chương đánh giá.

4.8 Mô hình CatBoost

CatBoost [13] là thuật toán gradient boosting xử lý đặc trưng phân loại bên trong cây, dùng trọng số tự động cân bằng lớp `auto_class_weights = "Balanced"` để đối phó với dữ liệu mất cân bằng mà không cần điều chỉnh thủ công. Không gian siêu tham số được khảo sát được trình bày trong Bảng 4.6.

Tham số	Giá trị khảo sát
iterations	số vòng huấn luyện, từ 300 đến 1 200
depth	độ sâu tối đa, từ 4 đến 10
learning_rate	tốc độ học, từ 0,005 đến 0,15
l2_leaf_reg	hệ số chính quy hóa L2, từ 0,1 đến 10
bagging_temperature, random_strength	mức ngẫu nhiên khi lấy mẫu và khi chọn ranh giới phân hoạch
border_count	số ngưỡng phân hoạch liên tục, từ 32 đến 255
min_data_in_leaf	số mẫu tối thiểu của một nút lá, từ 1 đến 20

Bảng 4.6: Không gian siêu tham số của CatBoost

4.9 Kết hợp mô hình bằng Stacking

Sau khi tinh chỉnh, ba cấu hình có điểm F2-Score trên tập Validation cao nhất được chọn làm mô hình cơ sở (base learners), gồm XGBoost–trọng số, LightGBM và CatBoost. Các mô hình cơ sở được kết hợp bằng StackingClassifier [14]: mỗi mô hình cơ sở sinh xác suất gian lận trên các fold cross-validation 3 lớp, xác suất này được đưa vào bộ tổng hợp (meta-learner) là một hồi quy Logistic có trọng số lớp cân bằng để học cách kết hợp các mô hình sao cho F2-Score cao nhất. Bước kết hợp này giúp mô hình tận dụng điểm mạnh từng mô hình và giảm phụ thuộc vào một cấu hình duy nhất.

4.10 Quá trình tinh chỉnh siêu tham số

Năm cấu hình Random Forest, XGBoost–trọng số, XGBoost–SMOTE nhẹ, LightGBM và CatBoost đều được tinh chỉnh bằng RandomizedSearchCV [15] với cross-validation 5-fold trên các đặc trưng đã mã hóa bằng TargetEncoder. Hàm tính điểm là `average_precision`, tương ứng với Average Precision (AP), một thước đo dựa trên đường Precision–Recall nhằm thích ứng với dữ liệu mất cân bằng [16]. Số lần thử ngẫu nhiên là 80 cho Random Forest, 80 cho mỗi cấu hình XGBoost, 60 cho LightGBM và 40 cho CatBoost, với `random_state = 42` để kết quả tái lập được. Kết quả cross-validation của từng cấu hình được trình bày trong Bảng 4.7. LightGBM với `scale_pos_weight` vượt trội rõ rệt so với bốn cấu hình còn lại, dẫn đầu về PR-AUC với 0,8525.

Quy trình tinh chỉnh trên được cố định cho toàn bộ sáu thử nghiệm ở chương đánh giá; giữa các thử nghiệm chỉ có tập đặc trưng đầu vào thay đổi, nhờ đó mọi so sánh đều không bị ảnh hưởng bởi khác biệt về ngân sách tìm kiếm siêu tham số.

Cấu hình	Số lần thử	Điểm PR-AUC tốt nhất (5-fold)
Random Forest	80	0,7555
XGBoost (Scale Pos Weight)	80	0,7973
XGBoost (Light SMOTE)	80	0,7642
LightGBM (Scale Pos Weight)	60	0,8525
CatBoost (Auto class weights)	40	0,7883

Bảng 4.7: Số lần thử và điểm PR-AUC cross-validation tốt nhất của từng cấu hình

5.1 Phạm vi sáu thử nghiệm

Để đánh giá ảnh hưởng của cách biểu diễn đặc trưng và của việc loại bỏ đặc trưng nhiễu, đề tài thiết kế sáu thử nghiệm. Tất cả thử nghiệm dùng chung một cách chia tập dữ liệu (tập Fit 19 236 mẫu – Validation 4 810 mẫu – tập kiểm tra 6 012 mẫu), cùng một bộ thuật toán và cùng một quy trình tinh chỉnh siêu tham số; chỉ có tập đặc trưng đầu vào là thay đổi. Ngưỡng quyết định của mỗi cấu hình được chọn trên tập Validation theo F2-Score với Precision tối thiểu 0,10, rồi mới đánh giá một lần duy nhất trên tập kiểm tra gồm 6 012 giao dịch, trong đó có 27 giao dịch gian lận và 5 985 giao dịch hợp lệ.

Bên cạnh các chỉ số đã trình bày ở phần trước, mỗi bảng kết quả dưới đây báo cáo thêm hai tỷ lệ lỗi phản ánh trực tiếp rủi ro của hệ thống. Tỷ lệ bỏ sót gian lận, viết tắt FNR, là tỷ lệ giao dịch gian lận thật không bị phát hiện; tỷ lệ báo nhầm giao dịch hợp lệ, viết tắt FPR, là tỷ lệ giao dịch hợp lệ bị cảnh báo nhầm là gian lận. Sáu thử nghiệm được tóm tắt trong Bảng 5.1.

Thử nghiệm	Cách biểu diễn đặc trưng	Số đặc trưng	Mô hình được huấn luyện
TN1	Sáu đặc trưng gốc, chưa tạo đặc trưng mới	6	Random Forest, XGBoost
TN2	OneHotEncoder cho đặc trưng phân loại	26	CatBoost, LightGBM, XGBoost, Random Forest
TN3	TargetEncoder cho đặc trưng phân loại	14	LightGBM, Stacking, Random Forest, XGBoost, CatBoost
TN4	TN3 trừ is_night, amt_vs_card_median, distance_km, amt	10	Stacking, LightGBM, XGBoost, Random Forest, CatBoost
TN5	TN3 trừ is_night, amt_vs_card_median, distance_km	11	LightGBM, XGBoost, Random Forest, CatBoost
TN6	TN3 trừ amt, amt_vs_card_median, distance_km	11	XGBoost, LightGBM, CatBoost, Random Forest

Bảng 5.1: Sáu thử nghiệm đánh giá ảnh hưởng của cách mã hóa và của việc loại bỏ đặc trưng

Ba thử nghiệm đầu (TN1–TN3) giữ nguyên tập đặc trưng và chỉ thay đổi cách biểu diễn,

nhằm tách ảnh hưởng của phương pháp mã hóa. Ba thử nghiệm sau (TN4–TN6) lấy TN3 làm chuẩn và loại bỏ dần các đặc trưng mà phân tích khám phá cho thấy chứa nhiễu, nhằm kiểm tra liệu việc giảm số chiều có làm mô hình tốt hơn hay không.

5.2 Kết quả từng thử nghiệm

5.2.1 Thử nghiệm 1: sáu đặc trưng gốc

Khi chỉ dùng sáu đặc trưng gốc mà không tạo thêm đặc trưng, hiệu quả của các mô hình giảm mạnh. Random Forest đạt PR-AUC 0,5016 và F1-Score 0,5319, trong khi XGBoost thấp hơn với PR-AUC 0,4141. Nguyên nhân là các đặc trưng thô chưa diễn đạt được mối quan hệ phi tuyến giữa số tiền giao dịch và hành vi của chủ thẻ — ví dụ số tiền so với mức trung bình của chính thẻ đó — là những tín hiệu mạnh nhất trong dữ liệu. Đây là cơ sở để đề tài bổ sung nhóm đặc trưng kỹ thuật ở phần trước.

Mô hình	Ngưỡng	Precision	Recall	F1	F2	ROC-AUC	PR-AUC	FNR	FPR
Random Forest	0,1464	0,5682	0,5000	0,5319	0,5123	0,9261	0,5016	0,5000	0,0043
XGBoost	0,0422	0,3506	0,5400	0,4252	0,4874	0,9567	0,4141	0,4600	0,0112

Bảng 5.2: Kết quả TN1 trên tập kiểm tra với sáu đặc trưng gốc

5.2.2 Thử nghiệm 2: mã hóa One-Hot

Mã hóa OneHotEncoder làm số đặc trưng tăng lên 26 và hiệu quả tăng vượt bậc so với TN1. Tuy nhiên đây chưa phải cấu hình tốt nhất: LightGBM đạt PR-AUC 0,8092 với F2-Score 0,7554, trong khi CatBoost đạt F2-Score cao hơn là 0,7801 nhờ Recall 0,8148, đổi lại Precision chỉ 0,6667. Điểm đáng chú ý là thời gian tinh chỉnh siêu tham số của LightGBM trong cấu hình này lên tới 1 143,26 giây, cao hơn gấp đôi so với các thử nghiệm sau, do không gian mã hóa rời rạc làm tìm kiếm chậm hơn.

Mô hình	Ngưỡng	Precision	Recall	F1	F2	ROC-AUC	PR-AUC	FNR	FPR
CatBoost	0,6186	0,6667	0,8148	0,7333	0,7801	0,9673	0,7821	0,1852	0,0018
LightGBM	0,0553	0,6774	0,7778	0,7241	0,7554	0,9756	0,8092	0,2222	0,0017
XGBoost	0,2858	0,7143	0,7407	0,7273	0,7353	0,9721	0,7693	0,2593	0,0013
Random Forest	0,2100	0,7600	0,7037	0,7308	0,7143	0,9727	0,7765	0,2963	0,0010

Bảng 5.3: Kết quả TN2 trên tập kiểm tra với mã hóa OneHotEncoder (26 đặc trưng)

5.2.3 Thử nghiệm 3: mã hóa Target Encoder

Thay OneHotEncoder bằng TargetEncoder thu gọn tập đặc trưng còn 14 chiều và cho kết quả cân bằng nhất trong sáu thử nghiệm. LightGBM với scale_pos_weight đạt PR-AUC 0,8197, F1-Score 0,8163 và F2-Score 0,7692 — đều là giá trị cao nhất — với Precision 0,9091, Recall 0,7407, FPR chỉ 0,0003 và thời gian dự đoán 0,0361 giây. Stacking đứng thứ hai về PR-AUC (0,7820) và là mô hình duy nhất bắt được nhiều gian lận hơn (Recall 0,7778, tức bắt 21 trong 27 vụ), nhưng báo nhầm 9 giao dịch hợp lệ so với chỉ 2 giao dịch của LightGBM.

Mô hình	Ngưỡng	Precision	Recall	F1	F2	ROC-AUC	PR-AUC	FNR	FPR
LightGBM	0,0634	0,9091	0,7407	0,8163	0,7692	0,9828	0,8197	0,2593	0,0003
Stacking	0,9827	0,7000	0,7778	0,7368	0,7609	0,9598	0,7820	0,2222	0,0015
Random Forest	0,2498	0,6897	0,7407	0,7143	0,7299	0,9516	0,7475	0,2593	0,0015
XGBoost	0,2545	0,5882	0,7407	0,6557	0,7042	0,9758	0,7398	0,2593	0,0023
CatBoost	0,8407	0,3922	0,7407	0,5128	0,6289	0,9711	0,6895	0,2593	0,0052

Bảng 5.4: Kết quả TN3 trên tập kiểm tra với mã hóa TargetEncoder (14 đặc trưng)

5.2.4 Thử nghiệm 4: loại bỏ bốn đặc trưng

TN4 loại bỏ bốn đặc trưng gồm is_night, amt_vs_card_median, distance_km và amt, còn 10 đặc trưng. LightGBM đạt PR-AUC 0,8231, cao nhất trong toàn bộ sáu thử nghiệm, và F1-Score 0,8000 gần bằng TN3. Tuy nhiên F2-Score giảm xuống 0,7634, Precision giảm còn 0,8696, FPR tăng gấp rưỡi lên 0,0005, thời gian dự đoán tăng lên 0,1048 giây và thời gian tính chỉnh tăng vọt lên 2 235,59 giây. Việc loại bỏ amt làm mô hình mất tín hiệu số tiền tuyệt đối, vốn là đặc trưng có ảnh hưởng lớn nhất theo phân tích mức độ ảnh hưởng của các đặc trưng trình bày tại Hình 5.1.

Mô hình	Ngưỡng	Precision	Recall	F1	F2	ROC-AUC	PR-AUC	FNR	FPR
Stacking	0,9910	0,7500	0,7778	0,7636	0,7721	0,9705	0,7819	0,2222	0,0012
LightGBM	0,0480	0,8696	0,7407	0,8000	0,7634	0,9826	0,8231	0,2593	0,0005
XGBoost	0,5840	0,9048	0,7037	0,7917	0,7364	0,9808	0,7338	0,2963	0,0003
Random Forest	0,1387	0,5882	0,7407	0,6557	0,7042	0,9673	0,7416	0,2593	0,0023
CatBoost	0,8407	0,3922	0,7407	0,5128	0,6289	0,9711	0,6895	0,2593	0,0052

Bảng 5.5: Kết quả TN4 trên tập kiểm tra sau khi loại bỏ bốn đặc trưng (10 đặc trưng)

5.2.5 Thử nghiệm 5: loại bỏ is_night, amt_vs_card_median, distance_km

TN5 giữ lại amt nhưng loại bỏ is_night, amt_vs_card_median và distance_km, còn 11 đặc trưng. LightGBM đạt PR-AUC 0,8205, gần bằng TN3, nhưng Recall giảm mạnh xuống 0,6667 nên F1-Score (0,7660) và F2-Score (0,7031) đều thấp hơn đáng kể, kéo theo FNR tăng lên 0,3333 tức bỏ sót 9 trong 27 giao dịch gian lận. Trong bốn thử nghiệm còn lại, XGBoost và CatBoost đều có PR-AUC dưới 0,76 và thấp hơn đáng kể so với LightGBM.

Mô hình	Ngưỡng	Precision	Recall	F1	F2	ROC-AUC	PR-AUC	FNR	FPR
LightGBM	0,1840	0,9000	0,6667	0,7660	0,7031	0,9845	0,8205	0,3333	0,0003
XGBoost	0,2234	0,4884	0,7778	0,6000	0,6954	0,9721	0,7584	0,2222	0,0037
Random Forest	0,1266	0,5405	0,7407	0,6250	0,6897	0,9478	0,7465	0,2593	0,0028
CatBoost	0,4779	0,6333	0,7037	0,6667	0,6884	0,9754	0,7816	0,2963	0,0018

Bảng 5.6: Kết quả TN5 trên tập kiểm tra sau khi loại bỏ ba đặc trưng (11 đặc trưng)

5.2.6 Thử nghiệm 6: loại bỏ amt, amt_vs_card_median, distance_km

TN6 cũng còn 11 đặc trưng nhưng giữ lại is_night thay cho amt. LightGBM đạt PR-AUC 0,8068, thấp hơn TN3 và là cấu hình LightGBM yếu nhất trong năm thử nghiệm; F2-Score 0,7364 và FNR 0,2963 đều kém hơn. XGBoost trong TN6 đạt F1 và F2 bằng nhau là 0,7407 do ngưỡng 0,2600 đặt trùng vị trí phân loại, cho thấy hiệu quả của XGBoost phụ thuộc mạnh vào tập đặc trưng hơn LightGBM.

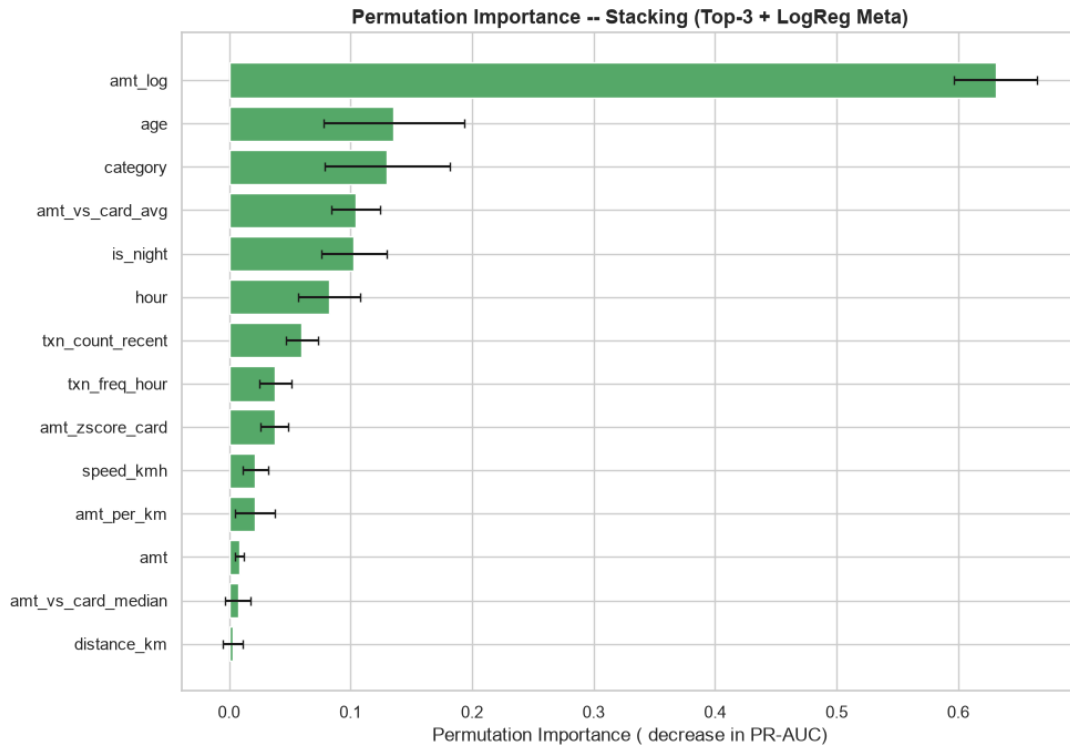
Mô hình	Ngưỡng	Precision	Recall	F1	F2	ROC-AUC	PR-AUC	FNR	FPR
XGBoost	0,2600	0,7407	0,7407	0,7407	0,7407	0,9628	0,7480	0,2593	0,0012
LightGBM	0,2666	0,9048	0,7037	0,7917	0,7364	0,9808	0,8068	0,2963	0,0003
CatBoost	0,9023	0,8571	0,6667	0,7500	0,6977	0,9754	0,7579	0,3333	0,0005
Random Forest	0,1321	0,5405	0,7407	0,6250	0,6897	0,9461	0,7297	0,2593	0,0028

Bảng 5.7: Kết quả TN6 trên tập kiểm tra sau khi loại bỏ ba đặc trưng (11 đặc trưng)

5.3 Giải thích ảnh hưởng của các đặc trưng

Để giải thích vì sao các thử nghiệm loại bỏ đặc trưng lại cho kết quả như vậy, đề tài đo mức độ ảnh hưởng của từng đặc trưng theo phương pháp hoán đổi ngẫu nhiên, thực hiện bằng hàm permutation_importance của thư viện scikit-learn [17]. Cụ thể, mỗi đặc trưng được hoán đổi ngẫu nhiên 10 lần và đo mức giảm PR-AUC, lấy trung bình trên 10 lần lặp. Phương pháp

này đáng tin cậy hơn chỉ số độ quan trọng sẵn có của mô hình cây vì đo trực tiếp ảnh hưởng thực tế lên chất lượng dự đoán, thay vì chỉ đo mức độ chia nút. Hình 5.1 trình bày kết quả cho mô hình Stacking tại cấu hình tốt nhất của TN3.



Hình 5.1: Mức độ ảnh hưởng của 14 đặc trưng theo phương pháp hoán đổi ngẫu nhiên, đo trên tập Validation

Hình 5.1 cho thấy `amt_log` áp đảo với mức ảnh hưởng 0,6309, lớn hơn tổng của mười đặc trưng còn lại; kế đến là `age` (0,1357), `category` (0,1302), `amt_vs_card_avg` (0,1042) và `is_night` (0,1029). Ở đầu bảng thấp là `amt` (0,0100), `amt_vs_card_median` (0,0046) và `distance_km` (-0,0013), tức gần như không đóng góp. Kết quả này giải thích trực tiếp cho toàn bộ sáu thử nghiệm:

- Vì `amt_log` là biến dẫn xuất từ `amt`, nên khi TN4 loại bỏ `amt` thì đặc trưng mạnh nhất cũng biến mất, giải thích việc F2-Score và Precision của TN4 đều thấp hơn TN3.
- Hai đặc trưng `distance_km` và `amt_vs_card_median` có ảnh hưởng gần bằng không, nên việc loại bỏ chúng ở TN5 và TN6 không mang lại lợi ích rõ ràng — đồng thời cho thấy chúng chỉ gây nhiễu nhẹ chứ không phải nguồn nhiễu nghiêm trọng.
- `is_night` có ảnh hưởng đáng kể (0,1029), nên TN5 loại bỏ nó đã làm Recall giảm từ 0,7407 xuống 0,6667.
- Ba đặc trưng `is_night`, `amt_vs_card_median` và `distance_km` mà ba thử nghiệm cuối loại bỏ có tổng ảnh hưởng chỉ khoảng 0,106, thấp hơn nhiều so với `amt_log`; đây là lý do kết

luận rằng việc loại bỏ đặc trưng không làm mô hình tốt hơn là có cơ sở định lượng chứ không chỉ là quan sát thực nghiệm.

5.4 Tổng hợp và so sánh sáu thử nghiệm

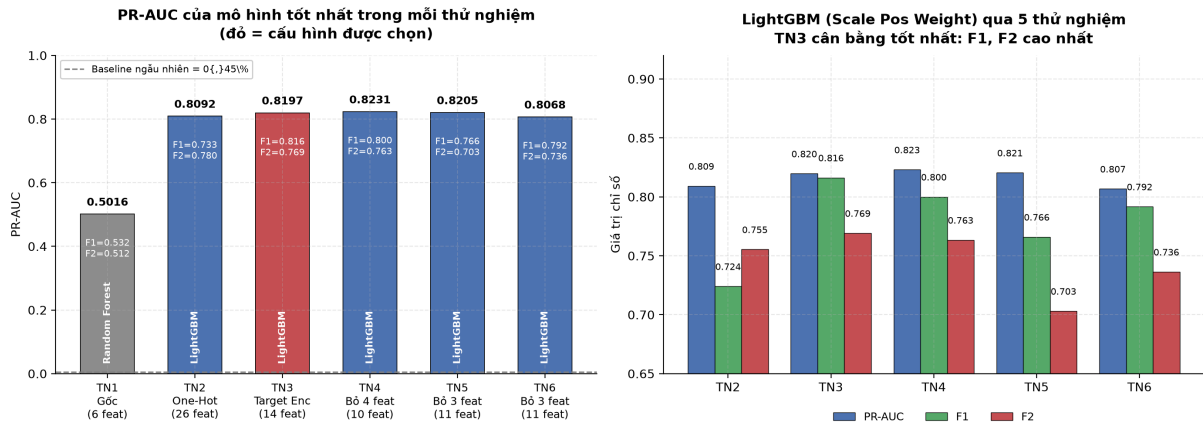
Bảng 5.8 và Hình 5.2 tổng hợp cấu hình tốt nhất của từng thử nghiệm. Ba kết luận nổi bật:

- Kỹ thuật tạo đặc trưng quan trọng hơn phương pháp mã hóa. So sánh TN1 với TN3 cho thấy PR-AUC tăng từ 0,5016 lên 0,8197, tức tăng hơn 63 phần trăm, chỉ nhờ bổ sung nhóm đặc trưng kỹ thuật. Trong khi đó, thay OneHotEncoder bằng TargetEncoder (TN2 → TN3) chỉ cải thiện PR-AUC từ 0,8092 lên 0,8197 nhưng lại giảm số chiều từ 26 xuống 14, giảm thời gian tinh chỉnh từ 1 143,26 giây xuống 231,85 giây và tăng Precision từ 0,6774 lên 0,9091.
- Loại bỏ đặc trưng không làm mô hình tốt hơn. Cả ba thử nghiệm TN4–TN6 đều có PR-AUC của LightGBM cao hơn TN3 một chút (0,8231; 0,8205; 0,8068) nhưng đều tệ hơn ở F1-Score hoặc F2-Score, đồng thời tốn thời gian dự đoán và tinh chỉnh nhiều hơn. Chênh lệch PR-AUC giữa TN3 và TN4 chỉ là 0,0034, nhỏ hơn nhiều so với sai số do chỉ có 27 giao dịch gian lận trong tập kiểm tra: đôi số dự đoán đúng của một điểm Recall là $1/27 \approx 0,037$. Vì vậy, việc loại bỏ đặc trưng không mang lại lợi ích có ý nghĩa, và tập 14 đặc trưng của TN3 được giữ lại.
- LightGBM là thuật toán bền vững nhất. Trong năm thử nghiệm có LightGBM, mô hình này dẫn đầu về PR-AUC ở cả năm thử nghiệm, đồng thời luôn giữ FPR ở mức 0,0003–0,0017, thấp hơn hẳn CatBoost (tới 0,0052) và XGBoost (tới 0,0037). Ngược lại, hiệu quả của XGBoost, CatBoost và Random Forest biến động mạnh theo tập đặc trưng: CatBoost rơi từ PR-AUC 0,7821 ở TN2 xuống 0,6895 ở TN3 và TN4; XGBoost giảm từ 0,7693 xuống 0,7398.

TN	Cấu hình tốt nhất	Số đặc trưng	Precision	Recall	F1	F2	PR-AUC	Thời gian (s)
TN1	Random Forest	6	0,5682	0,5000	0,5319	0,5123	0,5016	1,8849
TN2	LightGBM	26	0,6774	0,7778	0,7241	0,7554	0,8092	0,1349
TN3	LightGBM	14	0,9091	0,7407	0,8163	0,7692	0,8197	0,0361
TN4	LightGBM	10	0,8696	0,7407	0,8000	0,7634	0,8231	0,1048
TN5	LightGBM	11	0,9000	0,6667	0,7660	0,7031	0,8205	0,0941
TN6	LightGBM	11	0,9048	0,7037	0,7917	0,7364	0,8068	0,0499

Bảng 5.8: Cấu hình tốt nhất của từng thử nghiệm trên tập kiểm tra. Với TN1, cột thời gian ghi thời gian huấn luyện cuối cùng thay vì thời gian dự đoán, do đó không so sánh trực tiếp với năm thử nghiệm còn lại.

Tổng hợp kết quả 6 thử nghiệm trên tập kiểm tra

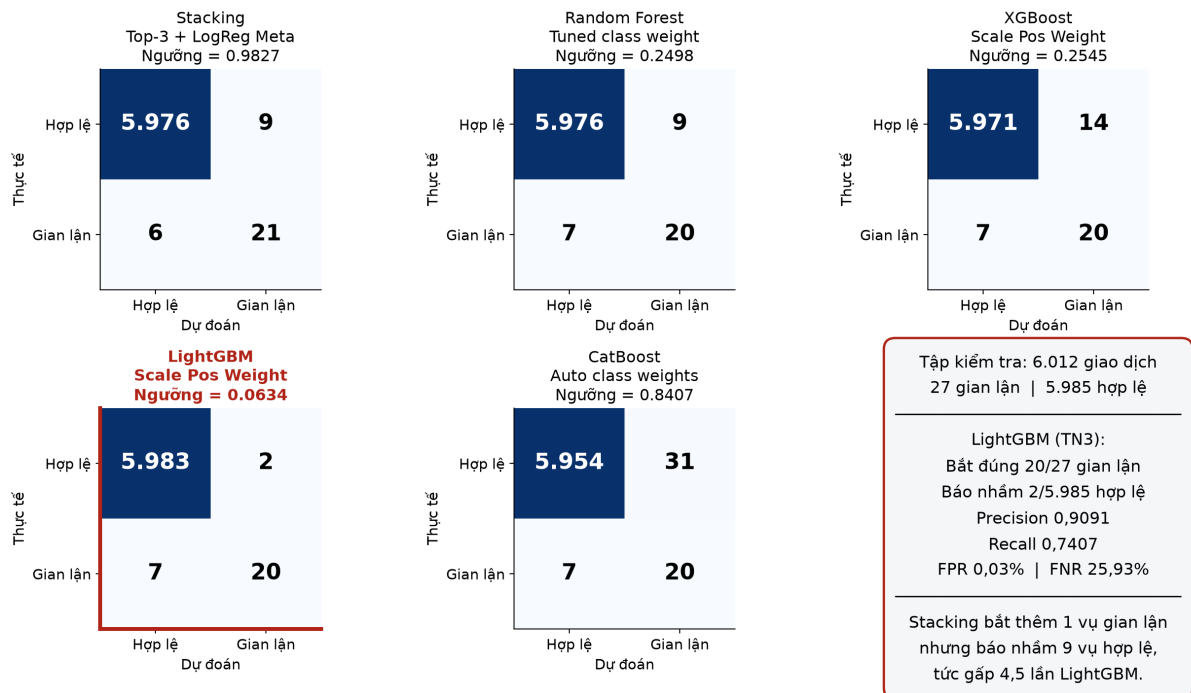


Hình 5.2: So sánh PR-AUC của cấu hình tốt nhất trong sáu thử nghiệm (trái) và hiệu quả của LightGBM qua năm thử nghiệm (phải)

5.5 Ma trận nhầm lẫn và đường Precision–Recall

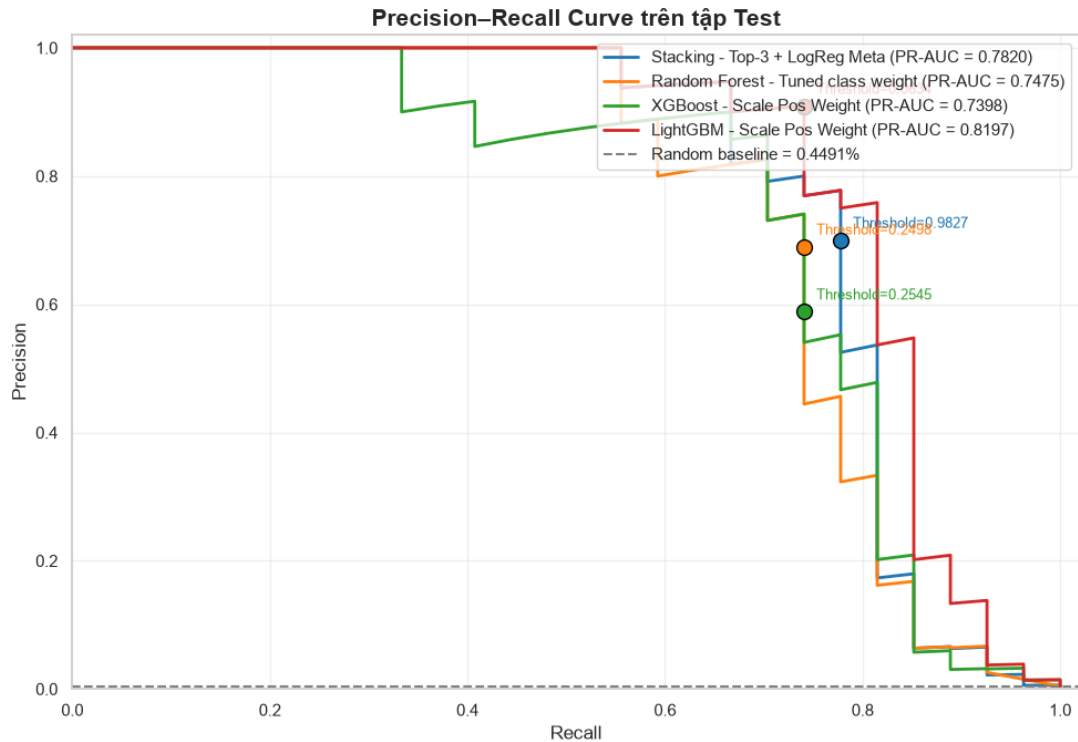
Hình 5.3 trình bày ma trận nhầm lẫn của năm mô hình tại cấu hình tốt nhất của TN3 trên tập kiểm tra. LightGBM phát hiện đúng 20 trong 27 giao dịch gian lận, bỏ sót 7, đồng thời chỉ báo nhầm 2 giao dịch hợp lệ — con số thấp nhất trong năm mô hình. Stacking bắt được nhiều gian lận hơn là 21 trong 27 nhưng phải chấp nhận 9 giao dịch hợp lệ bị báo nhầm, tức gấp 4,5 lần so với LightGBM trong khi chỉ bắt thêm được 1 vụ gian lận. Ba mô hình còn lại cùng bắt 20 vụ nhưng báo nhầm lần lượt 9, 14 và 31 giao dịch hợp lệ. Xét theo tỷ lệ, FPR của LightGBM là 0,03% trên 5 985 giao dịch hợp lệ, tức thấp hơn tỷ lệ gian lận quan sát được là 0,45%.

Ma trận nhầm lẫn trên tập kiểm tra tại cấu hình tốt nhất của TN3



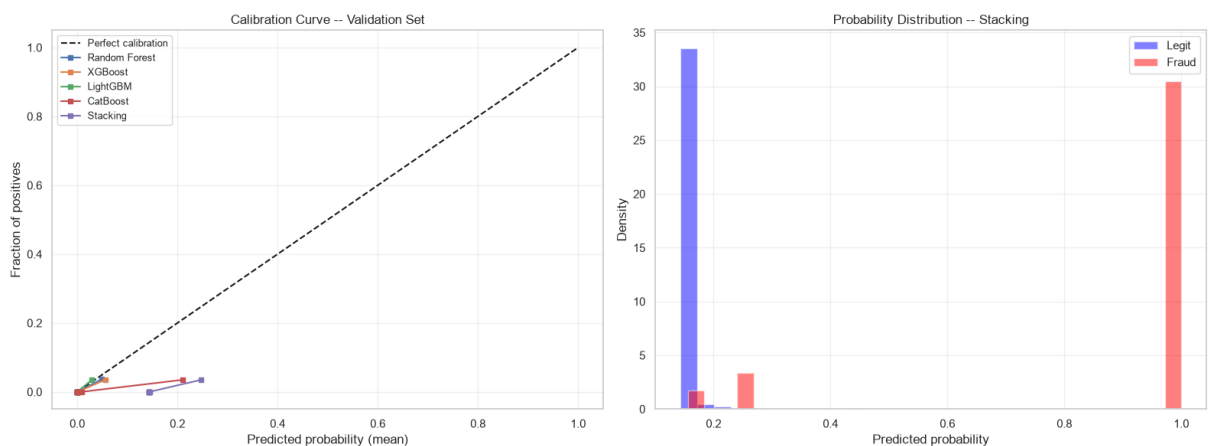
Hình 5.3: Ma trận nhầm lẫn của các mô hình tại cấu hình tốt nhất của TN3 trên tập kiểm tra

Hình 5.4 vẽ đường Precision–Recall của bốn mô hình trên tập kiểm tra. Đường của LightGBM nằm cao hơn các đường còn lại trên hầu hết lân vùng thao tác thực tế, đạt PR-AUC 0,8197, tiếp theo là Stacking 0,7820, Random Forest 0,7475 và XGBoost 0,7398. Điểm hoạt động của LightGBM tại ngưỡng 0,0634 nằm ở vùng Recall cao với Precision 0,9091, tức độ chính xác cao gấp khoảng 202 lần so với dự đoán ngẫu nhiên. Đường nét đứt ngang là đường cơ sở ứng với tỷ lệ gian lận trong tập kiểm tra là 0,4491%; toàn bộ các mô hình nằm trên đường này ở vùng ngưỡng đã chọn, xác nhận mô hình có giá trị thực tế.



Hình 5.4: Đường Precision–Recall của các mô hình tại cấu hình tốt nhất của TN3 trên tập kiểm tra

Hình 5.5 bổ sung hai góc nhìn về chất lượng xác suất dự đoán. Đường hiệu chuẩn trên tập Validation cho thấy xác suất dự đoán của các mô hình thấp hơn đáng kể so với tần suất thực tế, một hạn chế chung của mô hình trên dữ liệu cực mất cân bằng. Biểu đồ phân phối xác suất của mô hình Stacking cho thấy hai lớp tách biệt khá rõ: giao dịch hợp lệ tập trung quanh 0,15 còn giao dịch gian lận tập trung quanh 0,98.



Hình 5.5: Đường hiệu chuẩn trên tập Validation và phân phối xác suất của mô hình Stacking

5.6 Lựa chọn mô hình phù hợp với bộ dữ liệu

Dựa trên sáu thử nghiệm, đề tài chọn LightGBM với `scale_pos_weight` trên tập 14 đặc trưng của TN3 làm mô hình đề xuất cuối cùng. Lý do chọn được tóm tắt như sau:

- Cân bằng tốt nhất trên ba chỉ số chính. LightGBM tại TN3 đồng thời đạt F1-Score cao nhất (0,8163) và F2-Score cao nhất (0,7692) trong toàn bộ sáu thử nghiệm, với PR-AUC 0,8197 thuộc nhóm cao nhất. Như Bảng 5.8 cho thấy, không mô hình nào khác đạt đồng thời hai chỉ số F1 và F2 ở mức cao.
- Chi phí nghiệp vụ thấp nhất. Với Precision 0,9091, trong 22 giao dịch được cảnh báo có 20 là gian lận thật, tức chỉ 2 giao dịch hợp lệ bị kiểm tra lại không cần thiết — tương đương FPR 0,0003. Đây là mức thấp nhất trong năm mô hình tại TN3 và bằng một nửa của Stacking. Trong hệ thống thực tế, mỗi giao dịch hợp lệ bị chặn nhầm đều sinh chi phí xác minh và khiếu nại, nên đây là ưu thế quan trọng.
- Chi phí tính toán thấp. Thời gian dự đoán toàn bộ 6 012 giao dịch là 0,0361 giây, nhanh nhất trong sáu thử nghiệm và thấp hơn TN4 tới 2,9 lần; thời gian tinh chỉnh siêu tham số là 231,85 giây, giảm 4,9 lần so với TN2 và 9,6 lần so với TN4.
- Không đánh đổi nhiều Recall để lấy PR-AUC. TN4 có PR-AUC cao hơn 0,0034 nhưng F2-Score thấp hơn 0,0058, FPR cao hơn và thời gian dự đoán gấp 2,9 lần. Với chỉ 27 mẫu gian lận trong tập kiểm tra, chênh lệch 0,0034 tương đương dưới 0,1 điểm phần trăm và nằm trong biên sai số, không đủ căn cứ để đánh đổi.

Về hướng cải thiện Recall, vì bản chất nghiệp vụ ưu tiên bắt gian lận hơn là giảm báo nhầm, nếu hệ thống cần Recall cao hơn thì nên hạ ngưỡng quyết định của LightGBM xuống dưới 0,0634 thay vì đổi sang mô hình khác: theo đường Precision–Recall tại Hình 5.4, hạ ngưỡng làm Recall tăng nhanh hơn đáng kể so với đổi mô hình, trong khi Precision giảm chậm. Ngưỡng vận hành cụ thể nên được chọn theo chi phí tương đối giữa một vụ gian lận bị bỏ sót và một giao dịch hợp lệ bị chặn nhầm, thay vì cố định theo F2-Score.

5.7 Đánh giá tính khả thi và hạn chế

5.7.1 Điểm mạnh

- Quy trình hoàn chỉnh từ nạp dữ liệu, làm sạch, tạo đặc trưng đến huấn luyện và đánh giá, sát với bài toán phân loại nhị phân mất cân bằng trong thực tế.

- Thiết kế sáu thử nghiệm có kiểm soát: ba thử nghiệm tách ảnh hưởng của phương pháp mã hóa, ba thử nghiệm còn lại kiểm tra ảnh hưởng của việc loại bỏ đặc trưng, nhờ đó kết luận về tầm quan trọng của kỹ thuật tạo đặc trưng và của TargetEncoder có cơ sở thực nghiệm thay vì chỉ dựa trên kinh nghiệm.
- Khảo sát đồng thời bốn họ thuật toán rừng ngẫu nhiên và gradient boosting (XGBoost, LightGBM, CatBoost) cùng bốn chiến lược xử lý mất cân bằng khác nhau, đồng thời kết hợp các mô hình tốt nhất bằng stacking, bảo đảm khách quan khi so sánh hiệu quả.
- Thiết kế quy trình đánh giá nghiêm ngặt: tách riêng tập Fit, Validation và tập kiểm tra; giới hạn giá trị bất thường và chuẩn hóa chỉ học từ tập huấn luyện, giúp tránh rò rỉ dữ liệu.
- Sử dụng bộ chỉ số phù hợp với dữ liệu mất cân bằng gồm F2-Score và PR-AUC thay vì Accuracy, kèm báo cáo riêng FNR và FPR để phản ánh trực tiếp hai loại rủi ro của hệ thống.

5.7.2 Hạn chế

- Bộ dữ liệu là dữ liệu mô phỏng với thời gian quan sát ngắn 10 ngày, chưa phản ánh đầy đủ các phương thức gian lận mới phát sinh theo thời gian.
- Tập kiểm tra chỉ chứa 27 giao dịch gian lận, nên mỗi giao dịch đúng thay đổi Recall khoảng 0,037. Chênh lệch PR-AUC giữa TN3 và TN4 là 0,0034, nhỏ hơn nhiều lần sai số này, khiến việc phân định thứ hạng giữa hai cấu hình chưa thật sự chắc chắn.
- Đặc trưng khoảng cách distance_km tạo ra từ vị trí địa lý chứa nhiễu do chất lượng dữ liệu tọa độ, có thể làm giảm hiệu quả của mô hình.
- Xác suất dự đoán của các mô hình chưa được hiệu chuẩn tốt trên tập Validation, ảnh hưởng đến việc diễn giải ngưỡng quyết định theo xác suất tuyệt đối.
- Các khoảng siêu tham số được khảo sát bằng tìm kiếm ngẫu nhiên với số lần thử có giới hạn, chưa khẳng định được cấu hình tối ưu toàn cục.

5.8 Hướng phát triển trong tương lai

- Mở rộng dữ liệu thực tế nhiều tháng và bổ sung các đặc trưng như dấu vân tay thiết bị, địa chỉ IP, lịch sử tranh chấp của thẻ.
- Thử thêm các tổ hợp loại bỏ đặc trưng khác, so sánh TargetEncoder với CatBoostEncoder và mã hóa theo chuỗi thời gian, nhằm cải thiện thêm Recall.

- Hiệu chuẩn xác suất dự đoán bằng CalibratedClassifierCV và chọn ngưỡng vận hành theo chi phí kinh doanh thay vì chỉ theo F2-Score.
- Áp dụng các kỹ thuật nâng cao như điều chỉnh trọng số lớp theo chi phí nghiệp vụ, học sâu trên chuỗi giao dịch, hoặc tinh chỉnh siêu tham số có hệ thống bằng tối ưu hóa Bayes.
- Mở rộng stacking với nhiều mô hình cơ sở và bộ tổng hợp khác nhau.
- Triển khai hệ thống chấm điểm theo thời gian thực với giám sát độ trễ và phát hiện lệch phân bố dữ liệu sau khi đưa vào vận hành.

Trong khuôn khổ môn học Trí tuệ nhân tạo, đề tài đã thiết lập một quy trình hoàn chỉnh để xây dựng hệ thống phát hiện giao dịch tài chính bất thường và nghi vấn gian lận, từ khâu nạp và làm sạch dữ liệu đến khâu thiết kế thử nghiệm, huấn luyện và đánh giá các mô hình học máy.

Đề tài đã xử lý bộ dữ liệu gồm 30 058 giao dịch thẻ tín dụng ghi nhận trong giai đoạn 21/06–30/06/2020, trong đó có 29 925 giao dịch hợp lệ và 133 giao dịch gian lận, chiếm 0,44% — tức lớp lớn gấp khoảng 225 lần lớp nhỏ. Qua quá trình làm sạch, tạo đặc trưng và tiền xử lý, 14 đặc trưng được chọn cho mô hình gồm các đặc trưng số tiền, tuổi, giờ giao dịch, tần suất và khoảng cách, tốc độ di chuyển, logarit số tiền, độ lệch chuẩn theo thẻ, cờ ban đêm và loại hình dịch vụ. Phân tích khám phá cho thấy gian lận tập trung ở một số loại dịch vụ nhất định, trong đó shopping_net, misc_net và grocery_pos chiếm 82/133 vụ, và có 104/133 vụ gian lận nằm trong vùng giá trị bất thường của cột số tiền giao dịch. Ba nhóm dịch vụ dẫn đầu chiếm 61,7% số vụ gian lận; kiểm định Chi-square cho thấy loại dịch vụ liên quan có ý nghĩa thống kê với gian lận ($p = 1,3 \times 10^{-23}$).

Về mô hình, đề tài đã huấn luyện Random Forest, XGBoost, LightGBM và CatBoost, kết hợp các chiến lược xử lý mất cân bằng gồm trọng số lớp, trọng số cho lớp thiểu số, trọng số tự động và SMOTE nhẹ, sau đó tích hợp ba mô hình tốt nhất bằng stacking, chọn ngưỡng quyết định tối ưu theo F2-Score trên tập Validation và đánh giá cuối cùng một lần duy nhất trên tập kiểm tra gồm 6 012 giao dịch, trong đó có 27 giao dịch gian lận. Sáu thử nghiệm được thiết kế nhằm tách ảnh hưởng của phương pháp mã hóa (sáu đặc trưng gốc, One-Hot với 26 đặc trưng, Target Encoder với 14 đặc trưng) và ảnh hưởng của việc loại bỏ đặc trưng (ba thử nghiệm còn lại với 10–11 đặc trưng).

Kết quả cho thấy kỹ thuật tạo đặc trưng đóng vai trò quyết định: khi chỉ dùng sáu đặc trưng gốc, PR-AUC chỉ đạt 0,5016, trong khi bổ sung nhóm đặc trưng kỹ thuật và mã hóa bằng TargetEncoder đưa PR-AUC lên 0,8197. So với OneHotEncoder, TargetEncoder vừa nâng cao hiệu quả vừa giảm số chiều từ 26 xuống 14, rút ngắn thời gian tinh chỉnh từ 1 143 giây xuống 232 giây. Ba thử nghiệm loại bỏ đặc trưng đều không làm mô hình tốt hơn, cho thấy các đặc trưng bị loại vẫn mang thông tin hữu ích hoặc chỉ tạo nhiễu nhỏ. Xét tổng thể sáu thử nghiệm, LightGBM là thuật toán bền vững nhất: mô hình dẫn đầu về PR-AUC ở cả năm thử nghiệm có

LightGBM, duy trì FPR ở mức 0,0003–0,0017 trong khi CatBoost và XGBoost biến động mạnh theo tập đặc trưng.

Cụ thể, LightGBM với `exttttscale_pos_weight` trên tập 14 đặc trưng mã hóa bằng `exttttTargetEncoder` (thử nghiệm 3) được chọn làm mô hình đề xuất cuối cùng. Mô hình đạt PR-AUC 0,8197, F1-Score 0,8163 và F2-Score 0,7692 — đều là giá trị cao nhất trong toàn bộ sáu thử nghiệm — với Precision 0,9091 và Recall 0,7407. Cụ thể hơn, trong 22 giao dịch được mô hình cảnh báo có 20 là gian lận thật, tức chỉ 2 giao dịch hợp lệ bị báo nhầm, tương đương tỷ lệ dương tính giả 0,03%; đổi lại mô hình bỏ sót 7 trong 27 giao dịch gian lận, tức 25,9%. Thời gian dự đoán toàn bộ 6 012 giao dịch chỉ 0,036 giây, nhanh nhất trong sáu thử nghiệm. So với phương án Stacking — mô hình bắt được nhiều gian lận hơn là 21 trong 27 vụ — LightGBM chỉ bỏ sót thêm 1 vụ gian lận nhưng báo nhầm ít hơn tới 7 giao dịch hợp lệ, phù hợp hơn với nguyên tắc ưu tiên giảm thiệt hại do kiểm tra nhầm. Cần lưu ý rằng phương án TN4 có PR-AUC cao hơn một chút là 0,8231, song chênh lệch 0,0034 này nhỏ hơn nhiều so với sai số do chỉ có 27 mẫu gian lận trong tập kiểm tra, trong khi TN4 lại kém hơn rõ rệt ở F2-Score, FPR và thời gian dự đoán.

Bên cạnh những kết quả đạt được, đề tài còn tồn tại một số hạn chế: bộ dữ liệu là dữ liệu mô phỏng với thời gian quan sát chỉ 10 ngày nên chưa phản ánh đầy đủ các phương thức gian lận mới; số lượng mẫu gian lận nhỏ khiến kết quả đánh giá nhạy cảm với từng vài giao dịch và chưa đủ để phân định chắc chắn thứ hạng giữa các cấu hình có PR-AUC gần nhau; đặc trưng khoảng cách `distance_km` chứa nhiều do chất lượng tọa độ; xác suất dự đoán chưa hiệu chuẩn tốt; và việc khảo sát siêu tham số mới dùng tìm kiếm ngẫu nhiên với số lần thử có giới hạn. Hướng phát triển trong tương lai bao gồm mở rộng nguồn dữ liệu thực tế và bổ sung đặc trưng hành vi thiết bị, thử thêm các phương án mã hóa và tổ hợp loại bỏ đặc trưng khác để cải thiện Recall, hiệu chuẩn xác suất và chọn ngưỡng vận hành theo chi phí kinh doanh, cũng như triển khai hệ thống giám sát theo thời gian thực.

Tóm lại, đề tài đã đạt được các mục tiêu đề ra: minh họa trọn vẹn quy trình ứng dụng trí tuệ nhân tạo vào bài toán phát hiện gian lận tài chính, đồng thời cung cấp cơ sở so sánh khách quan về hiệu quả của các thuật toán Random Forest, XGBoost, LightGBM, CatBoost và stacking trên cùng một bộ dữ liệu mô phỏng giao dịch thẻ tín dụng. Với tám mươi phần trăm giao dịch hợp lệ không bị chặn nhầm và thời gian dự đoán dưới 0,04 giây, mô hình LightGBM đề xuất đã cho thấy tiềm năng đủ để triển khai thử nghiệm trong hệ thống chấm điểm rủi ro giao dịch thời gian thực.

Tài liệu tham khảo

- [1] Siddhartha Bhattacharyya **and others**. “Data Mining for Credit Card Fraud: A Comparative Study”. in *Decision Support Systems*: 50.3 (2011), **pages** 602–613. DOI: [10.1016/j.dss.2010.08.008](https://doi.org/10.1016/j.dss.2010.08.008).
- [2] Haibo He **and** Eduardo A. Garcia. “Learning from Imbalanced Data”. in *IEEE Transactions on Knowledge and Data Engineering*: 21.9 (2009), **pages** 1263–1284. DOI: [10.1109/TKDE.2008.239](https://doi.org/10.1109/TKDE.2008.239).
- [3] Andrea Dal Pozzolo **and others**. “Calibrating Probability with Undersampling for Unbalanced Classification”. in *2015 IEEE Symposium Series on Computational Intelligence*: 2015, **pages** 159–166. DOI: [10.1109/SSCI.2015.33](https://doi.org/10.1109/SSCI.2015.33).
- [4] Jesse Davis **and** Mark Goadrich. “The Relationship between Precision-Recall and ROC Curves”. in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*: 2006, **pages** 233–240. DOI: [10.1145/1143844.1143874](https://doi.org/10.1145/1143844.1143874).
- [5] Wes McKinney. “Data Structures for Statistical Computing in Python”. in *Proceedings of the 9th Python in Science Conference*: Austin, Texas, 2010, **pages** 56–61.
- [6] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort **and others**. “Scikit-learn: Machine Learning in Python”. in *Journal of Machine Learning Research*: 12 (2011), **pages** 2825–2830.
- [7] Paula Branco, Luís Torgo **and** Rita P. Ribeiro. “A Survey of Predictive Modeling on Imbalanced Domains”. in *ACM Computing Surveys*: 49.2 (2016), **pages** 1–50. DOI: [10.1145/2907070](https://doi.org/10.1145/2907070).
- [8] Nitesh V. Chawla **and others**. “SMOTE: Synthetic Minority Over-sampling Technique”. in *Journal of Artificial Intelligence Research*: 16 (2002), **pages** 321–357. DOI: [10.1613/jair.953](https://doi.org/10.1613/jair.953).
- [9] Leo Breiman. “Random Forests”. in *Machine Learning*: 45.1 (2001), **pages** 5–32. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [10] Tianqi Chen **and** Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*: 2016, **pages** 785–794. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [11] Guolin Ke **and others**. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. in *Advances in Neural Information Processing Systems (NeurIPS)*: **volume** 30. 2017, **pages** 3146–3154.
- [12] LightGBM Developers. *LightGBM Documentation: Parameters*. URL: <https://lightgbm.readthedocs.io/en/latest/Parameters.html> ([urlseen](#) 23/09/2026).
- [13] Liudmila Prokhorenkova **and others**. “CatBoost: Unbiased Boosting with Categorical Features”. in *Advances in Neural Information Processing Systems (NeurIPS)*: **volume** 31. 2018, **pages** 6638–6648.
- [14] David H. Wolpert. “Stacked Generalization”. in *Neural Networks*: 5.2 (1992), **pages** 241–259. DOI: [10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1).
- [15] scikit-learn Developers. *scikit-learn: RandomizedSearchCV*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html ([urlseen](#) 23/09/2026).
- [16] scikit-learn Developers. *scikit-learn: average_precision_score*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html ([urlseen](#) 23/09/2026).
- [17] scikit-learn Developers. *scikit-learn: permutation_importance*. URL: https://scikit-learn.org/stable/modules/generated/sklearn.inspection.permutation_importance.html ([urlseen](#) 25/09/2026).